Systems Primer

CS Fundamentals for the Curious Engineer

Architecture · OS · Memory · Concurrency · Scale

Sahil Raj (After Dark)

A ground-up guide to the systems concepts every software engineer should know.
From transistors to distributed databases — explained with analogies,
code examples, and the intuition that textbooks often skip.

Contents

Chapter 1

Computer Architecture

*Understanding how a computer physically works — from transistors to caches — gives you an intuition for **why** code runs fast or slow, and why certain programming patterns matter at all. This chapter builds that foundation.*

1.1 The Big Picture

A modern computer is built from billions of tiny switches called **transistors**. When grouped together, transistors form **logic gates** (AND, OR, NOT), which in turn form **arithmetic circuits**, **memory cells**, and everything else you see.

At the highest level, a computer has four main components:

Core Components of a Computer

- **CPU (Central Processing Unit)** — executes instructions.
- **Memory (RAM)** — stores data and instructions currently in use.
- **Storage** — persistent data (SSD, HDD).
- **I/O Devices** — keyboard, screen, network, etc.

The CPU, memory, and I/O are connected by a **bus** — a set of electrical lines that carry data, addresses, and control signals.

The **Von Neumann architecture** (the design used by virtually every modern computer) stores both program instructions and data in the same memory. The CPU reads an instruction, executes it, and moves to the next — this is the stored-program model.

Analogy: The Office

Think of the CPU as a worker at a desk. The **RAM** is the desk surface — fast to access, limited space. The **disk** is a filing cabinet across the room — large but slow. **Registers** are items literally in the worker's hands — instant access, tiny capacity. The worker **fetches** documents from the cabinet to the desk, works on them, then files them back.

1.2 Inside the CPU

The CPU is divided into several functional units:

CPU Functional Units

- **ALU (Arithmetic Logic Unit)** — performs arithmetic (+, -, *, /) and logical (AND, OR, XOR, shifts) operations on data.
- **Control Unit (CU)** — decodes instructions and orchestrates the ALU, registers, and memory to execute them.
- **Register File** — a tiny bank of ultra-fast storage directly on the CPU chip. Typical CPUs have 16–32 general-purpose registers, each 64 bits wide.
- **Program Counter (PC)** — a special register holding the address of the *next instruction* to execute.
- **Instruction Register (IR)** — holds the instruction *currently* being decoded/executed.

Registers are the fastest storage in the system. Accessing a register takes a fraction of a nanosecond. Accessing RAM takes ∼100 ns — roughly 100× slower. This gap is the fundamental reason caches exist.

1.3 Instruction Set Architecture (ISA)

The **Instruction Set Architecture (ISA)** is the contract between hardware and software. It defines what instructions the CPU understands, what registers exist, how memory is addressed, and how exceptions work.

ISA

The ISA is the **abstract interface** of a CPU. Two CPUs with the same ISA can run the same binary, even if they are built completely differently internally. Examples: **x86-64** (Intel/AMD desktops/laptops), **ARM** (phones, Apple M-series), **RISC-V** (open-source, growing).

Each instruction in the ISA performs a primitive operation:

- **Data transfer**: `MOV rax, rbx` (copy register to register)
- **Arithmetic**: `ADD rax, 5` (add 5 to rax)
- **Logical**: `AND rax, rbx`
- **Control flow**: `JMP label`, `JNE label` (jump if not equal)
- **Memory**: `LOAD rax, [addr]`, `STORE [addr], rax`

What C Compiles To

```
// C source
int add(int a, int b) { return a + b; }

// Compiled x86-64 assembly (simplified)
add:
    mov eax, edi    ; move first argument (a) into eax
    add eax, esi    ; add second argument (b) to eax
    ret             ; return (eax holds return value)
```

Your C code is just a convenient way to write machine instructions.

1.4 The Fetch–Decode–Execute Cycle

The CPU repeats this cycle billions of times per second:

The Classic CPU Cycle

1. **Fetch** — Read the instruction at the address stored in the PC from memory into the IR. Increment the PC.
2. **Decode** — The control unit interprets the instruction bits: what operation, which registers, what immediate values.
3. **Execute** — The ALU performs the operation. Results go to registers or memory.
4. **Write-back** — Store the result back to the destination register or memory (on some architectures this is a distinct stage).

On a 3 GHz CPU, one cycle takes ≈ 0.33 ns. Simple instructions (like adding two registers) complete in 1 cycle. Memory accesses may stall the pipeline for 200+ cycles while waiting for RAM.

1.5 Instruction Pipelining

Executing one instruction completely before starting the next wastes hardware. Instead, modern CPUs **pipeline** instructions — like an assembly line.

Pipelining

While instruction N is in the Execute stage, instruction $N+1$ is being Decoded, and instruction $N+2$ is being Fetched simultaneously. Each stage runs in parallel, increasing **throughput** (instructions completed per second) without necessarily making any single instruction faster.

A 5-stage pipeline (Fetch, Decode, Execute, Memory, Write-back) can have 5 instructions in flight at once. A modern out-of-order CPU may have 15–20 pipeline stages.

Pipeline Visualization

```
Cycle:   1    2    3    4    5    6    7
Instr 1: F    D    E    M    W
Instr 2:      F    D    E    M    W
Instr 3:           F    D    E    M    W
```

F=Fetch, D=Decode, E=Execute, M=Memory access, W=Writeback
Without pipelining, 3 instructions take 15 cycles. With pipelining: 7 cycles.

1.6 Pipeline Hazards

Pipelines stall when an instruction depends on a result not yet produced. These situations are called **hazards**.

Types of Pipeline Hazards

- **Data Hazard** — Instruction $N+1$ needs a value computed by N, but N hasn't finished yet. Hardware inserts **bubbles** (stall cycles) or uses **forwarding** (passing the result directly before write-back).
- **Control Hazard** — A branch instruction changes the PC. The CPU doesn't know which instruction comes next until the branch is resolved, so several fetched instructions may be wrong.
- **Structural Hazard** — Two instructions need the same hardware resource simultaneously (rare in modern CPUs due to duplicated units).

Forwarding (or *bypassing*) is a hardware technique that routes a computed result directly from the output of one pipeline stage to the input of a later stage, avoiding stalls in many data hazard cases.

1.7 Branch Prediction

The **branch predictor** is a hardware component that guesses the outcome of conditional branches *before* they are resolved, allowing the pipeline to keep filling with (hopefully) the correct instructions.

Branch Prediction

For a branch like `if (x > 0) ...`, the predictor guesses whether the branch is "taken" (jump) or "not taken" (fall through) and speculatively fetches instructions along that path. If the guess is **correct**: no penalty. If **wrong**: the pipeline must be **flushed** (wasted work), costing typically 10–20 cycles.

Modern CPUs achieve >99% prediction accuracy on typical code using sophisticated statistical algorithms (e.g., two-level adaptive predictors, TAGE predictors).

Branch Misprediction in Hot Loops

Unpredictable branches inside tight loops are a common performance killer. Sorting an array before branching on its contents, or using branchless arithmetic (`result = (x > 0) ? a : b` compiled to a CMOV instruction), can give significant speedups when branch misprediction is the bottleneck.

1.8 Out-of-Order Execution

Modern CPUs don't execute instructions in the exact order written — they execute them in **data-dependency order**, keeping execution units busy whenever possible.

Out-of-Order Execution (OOO)

The CPU analyzes a **window** of upcoming instructions (often 100–300 instructions). Instructions whose operands are ready execute immediately, even if earlier instructions are still waiting for a cache miss or branch resolution. Results are **committed** (made permanent) in-order, preserving correctness.

OOO in Action

```
1 int a = memory[0];     // slow load -- cache miss
2 int b = 2 + 3;         // can execute NOW, doesn't depend on a
3 int c = b * 4;         // can execute after b is ready
4 int d = a + 1;         // must wait for the memory load
```

A sequential CPU stalls on line 1. An OOO CPU executes lines 2 and 3 while line 1 is still waiting for memory, then computes line 4 when the load completes.

1.9 RISC vs. CISC

RISC and CISC

- **RISC** (Reduced Instruction Set Computer) — few, simple, fixed-size instructions. Each instruction does one thing. Easier to pipeline. Examples: ARM, RISC-V, MIPS.
- **CISC** (Complex Instruction Set Computer) — many instructions of varying complexity and size. A single instruction can do a multi-step operation (e.g., "load from memory, add, store back"). Harder to pipeline directly, so modern CISC CPUs (x86) internally translate CISC instructions to RISC-like "micro-ops" before pipelining.

In practice, the distinction has blurred. x86 CPUs are as fast as ARM for server workloads, and Apple's ARM M-series chips beat x86 in performance-per-watt for laptop workloads. The ISA matters less than the microarchitectural implementation.

1.10 The Memory Hierarchy

Moving data between components costs time. The hierarchy exists because: *fast storage is expensive; cheap storage is slow.*

Memory Hierarchy (fastest to slowest)

Level	Typical Size	Latency	Location
Registers	32 × 64-bit	< 1 ns	On CPU core
L1 Cache	32–64 KB	~ 1 ns	On CPU core
L2 Cache	256 KB – 1 MB	~ 4 ns	On CPU core
L3 Cache	4–64 MB	~ 10 ns	Shared on chip
Main Memory (RAM)	8 – 256 GB	~100 ns	DIMM slots
NVMe SSD	1 – 8 TB	~100 µs	PCIe slot
HDD	1 – 20 TB	~10 ms	SATA

The CPU's caches are transparent to the programmer — you don't explicitly manage them. The hardware automatically moves data from RAM into cache when accessed, and evicts old data to make room. Understanding them, however, lets you write faster code.

1.11 CPU Caches: Hits, Misses, and Locality

Cache Hit and Cache Miss

- **Cache Hit** — the requested data is already in cache. Served in 1–10 ns.
- **Cache Miss** — the data is not in cache. The CPU must fetch it from a lower (slower) level. Miss penalty can be 100–10,000 ns.

Caches exploit two forms of **locality**:

- **Temporal locality** — data accessed recently is likely to be accessed again soon (e.g., a loop counter).
- **Spatial locality** — data near recently accessed data is likely to be accessed soon. Caches load an entire **cache line** (typically 64 bytes) on each miss.

Spatial Locality in Practice

```
// Fast: accesses memory sequentially -- one cache line = 8 doubles
for (int i = 0; i < N; i++) sum += arr[i];   // cache friendly

// Slow: jumps around memory -- constant cache misses
for (int i = 0; i < N; i++) sum += arr[rand() % N];  // cache unfriendly
```

On large arrays, the sequential version can be 10–50× faster due to caching.

1.12 Cache Associativity and Replacement

The cache is divided into **sets** and **ways**:

Cache Organisation

- **Direct-mapped** — each memory address maps to exactly one cache line slot. Simple, fast, but prone to *conflict misses* (two hot addresses fighting for the same slot).
- **N-way Set Associative** — each address can go into any of N slots within a set. Reduces conflict misses. Modern L1 is typically 4-way or 8-way.
- **Fully Associative** — any address can go anywhere. Maximum flexibility, expensive hardware (used in TLBs, small structures).

When a set is full and a new line must be loaded, the hardware evicts an existing line using a **replacement policy**: most commonly **LRU** (Least Recently Used) or a pseudo-LRU approximation.

Write policies determine what happens when the CPU writes data:

- **Write-through**: write goes to both cache and RAM immediately. Simple, consistent, but wastes bandwidth.
- **Write-back**: write only goes to cache. Memory is updated when the line is evicted. Faster, but requires a "dirty bit" to track modified lines. Used by virtually all modern CPUs.

1.13 Cache Coherence in Multi-Core Systems

Each CPU core typically has its own L1 and L2 cache, but shares L3 and RAM.

Cache Coherence

If Core 0 modifies a value in its L1 cache, Core 1's cached copy of the same address is now **stale**. Cache coherence protocols ensure that all cores see a **consistent view** of memory at all times.

The dominant protocol is **MESI** (Modified, Exclusive, Shared, Invalid): each cache line has a state tag. When one core modifies a line, other cores' copies are **invalidated** via messages on the interconnect. The next access on another core triggers a cache-to-cache transfer.

False Sharing

If two threads on different cores frequently write to **different variables that happen to be on the same cache line**, coherence traffic constantly invalidates each other's lines — even though they're touching different data. This is called **false sharing** and can devastate multi-threaded performance.
Fix: pad data structures so that variables modified by different threads land on different cache lines (align to 64 bytes).

1.14 Multi-Core, Hyper-Threading, and SIMD

Multi-Core CPUs

A modern CPU die contains multiple **cores** — each a full, independent CPU with its own registers, ALU, and L1/L2 caches. They share L3 cache and memory controller. A 16-core CPU can execute 16 independent instruction streams in parallel.

Hyper-Threading (Intel) / **Simultaneous Multi-Threading (SMT)**: Each physical core exposes *two logical cores* to the OS. Each logical core has its own register file and PC, but they share the execution units (ALU, FPU). When one logical core stalls (e.g., cache miss), the other can run. In practice, SMT gives 10–30% throughput improvement for mixed workloads.

SIMD — Single Instruction, Multiple Data

SIMD instructions operate on a **vector** of values simultaneously. For example, AVX2 (x86) operates on 256-bit registers, processing 8 floats at once.

```
// Conceptually, instead of:
for (int i=0; i<8; i++) c[i] = a[i] + b[i];  // 8 ADD instructions

// SIMD executes one instruction:
c[0..7] = a[0..7] + b[0..7];  // one 256-bit SIMD ADD
```

Compilers can auto-vectorize simple loops. Manual SIMD intrinsics give even more control.

1.15 Buses and I/O

The **system bus** connects the CPU, memory, and peripheral devices:

- **Address bus** — carries the memory address the CPU wants to read/write. A 64-bit address bus can address 2^{64} bytes (≈18 exabytes).
- **Data bus** — carries the actual data being transferred.
- **Control bus** — carries signals like Read/Write, Interrupt, Clock.

Modern systems use **PCIe** (Peripheral Component Interconnect Express) for high-speed devices (GPUs, NVMe SSDs). PCIe 4.0 ×16 provides ∼32 GB/s bandwidth — enough to saturate multiple NVMe drives simultaneously.

Chapter 1 — Key Takeaways

- The CPU repeatedly **fetches, decodes, and executes** instructions.
- **Pipelining** overlaps stages to increase throughput; **hazards** cause stalls.
- **Branch prediction** and **out-of-order execution** keep the pipeline busy.
- The **memory hierarchy** (registers → L1 → L2 → L3 → RAM) exists because fast storage is expensive.
- **Cache-friendly** code (sequential access, small working sets) can be 10–100× faster than cache-unfriendly code.
- **False sharing** silently kills multi-threaded performance.
- **SIMD** and **multi-core** parallelism are how modern CPUs achieve high throughput.

1.16 The Reorder Buffer and Tomasulo's Algorithm

Out-of-order execution needs hardware bookkeeping to commit results in-order.

Reorder Buffer (ROB)

The **ROB** is a circular queue of all in-flight instructions in program order. When an instruction is decoded, it gets an entry in the ROB. When it executes (possibly out of order), it writes its result to the ROB entry (not the register file). Instructions **commit** (retire) only from the **head** of the ROB — strictly in program order — at which point the result is written to the architectural register file.
This gives the illusion of in-order execution to software while executing out-of-order internally. If an exception occurs or a branch is mispredicted, everything in the ROB after the faulting instruction is simply **flushed** — no architectural state was modified.

Tomasulo's Algorithm (Register Renaming)

Register renaming eliminates false data dependencies (WAW, WAR hazards). The CPU has more physical registers than the ISA exposes (e.g., x86-64 exposes 16 but modern CPUs have 160–500 physical registers). Each instruction's destination is assigned a fresh physical register, eliminating write-after-write and write-after-read hazards. Only true read-after-write dependencies remain.
Reservation stations hold instructions waiting for their operands. When all operands are ready (snooping the common data bus), the instruction issues to an execution unit. Multiple execution units (multiple ALUs, load/store units, FPUs) execute independent instructions in parallel.

Modern OOO Pipeline (Simplified)

```
Fetch -> Decode -> Rename(ROB alloc) -> Dispatch(Reservation Stations)
      -> Issue(when ready) -> Execute(ALU/FPU/LSU) -> Writeback(to ROB)
      -> Commit(in-order, from ROB head)
```

An Intel Skylake core can have up to 224 instructions in flight simultaneously.

1.17 Load/Store Units and the Memory Order Buffer

Memory operations (loads and stores) are the most complex part of a modern CPU microarchitecture.

Load Queue and Store Queue

- **Store Queue**: holds store addresses and data until the store commits. Stores cannot write to cache until they commit (retire from ROB head), because a mispredicted branch could have speculatively generated the store.
- **Load Queue**: tracks in-flight loads. Loads can execute speculatively before stores before them commit — but the CPU must verify no earlier store wrote to the same address (**store-to-load forwarding** or detect the hazard and re-execute the load).

Store-to-load forwarding: if a load address matches a pending store address in the store queue, the value is forwarded directly — no cache access needed.

Memory Ordering Violations

If a load speculatively executes before an earlier store to the same address commits with a different value, the CPU detects a **memory order violation** and must re-execute the load and all dependent instructions. This is called a **machine clear** and can cost 50–150 cycles. High-performance code avoids aliasing patterns that trigger frequent machine clears (false dependency on the store-forwarding predictor).

1.18 Hardware Prefetching

Hardware Prefetcher

The CPU silently monitors memory access patterns and speculatively fetches cache lines into L1/L2 *before* they are requested.
Types of hardware prefetchers:

- **Stream prefetcher**: detects sequential or strided access patterns (e.g., iterating an array). Prefetches the next N cache lines ahead.
- **Stride prefetcher**: detects constant-stride access (e.g., every 4th element). Prefetches at the detected stride.
- **Spatial prefetcher**: when one cache line is accessed, prefetches adjacent lines from the same **spatial footprint** (same 2 KB region).
- **Temporal prefetcher** (newer CPUs): learns irregular access patterns using a history table.

Software Prefetch Hints

When the hardware prefetcher can't predict your access pattern (e.g., pointer chasing in a linked list), you can issue explicit prefetch instructions:

```
// GCC/Clang intrinsic
__builtin_prefetch(ptr->next, 0, 3);  // prefetch for read, high temporal locality
// x86 assembly equivalent: PREFETCHT0 [addr]

// In a linked-list traversal:
while (node) {
    __builtin_prefetch(node->next->next, 0, 1);  // prefetch 2 nodes ahead
    process(node);
    node = node->next;
}
```

Can reduce latency by overlapping computation with cache miss latency.

1.19 Write Combining and Store Buffers

Write Combining Buffer (WCB)

Between the CPU and the cache hierarchy are a small number (4–12) of **write combining buffers**. When the CPU writes to uncacheable memory (e.g., framebuffer, PCI device registers, or write-combining mapped memory), writes to the same cache line are **combined** in the WCB before being sent to the bus as a single burst write.
If you write bytes 0, 1, 2, ...63 of a line in order, one bus transaction is needed. If you write them out of order or mix reads and writes, WCBs flush early — many bus transactions, much slower.

Write Combining Matters for GPU Programming

GPU framebuffers and CUDA/OpenCL host-device transfers use write-combining memory.

```
// Slow: writes scattered, WCB flushes repeatedly
for (int i = 0; i < N; i++) framebuf[i * 64] = value;

// Fast: sequential writes fill WCB completely before flush
for (int i = 0; i < N; i++) framebuf[i] = value;
```

Linear writes can be 4–8x faster than strided writes to WC memory.

1.20 DRAM Internals: Banks, Rows, and Refresh

How DRAM Works

DRAM stores each bit as a charge in a tiny capacitor. Capacitors leak — DRAM must be **refreshed** every 64 ms (read and rewrite every row) or data is lost. This refresh takes the bank offline, causing **refresh stalls**. DRAM is organised as:

- **Banks**: independent memory arrays that can be accessed in parallel. A DIMM typically has 8–16 banks.
- **Rows** (pages): each bank has thousands of rows (~8 KB each).
- **Columns**: the bits within a row.

Access sequence: (1) Activate (open) the row — copies it to the row buffer (~15 ns). (2) Column read/write from the row buffer (~5 ns per access). (3) Precharge (close) the row before accessing a different one (~15 ns).

A **row buffer hit** (accessing the same row again) is very fast (~5 ns). A **row buffer miss** requires precharge + activate + read (~35 ns). A **bank conflict** requires switching banks (~50+ ns).

DRAM-Friendly Access Patterns

Access memory in **row-major** order (C arrays are row-major) to maximise row buffer hits. Matrix transpose is slow because column-major access causes constant row buffer misses. Tiled/blocked matrix algorithms (cache-oblivious algorithms) structure accesses to stay in the row buffer.

1.21 Power Management: P-states, C-states, and DVFS

Dynamic Voltage and Frequency Scaling (DVFS)

The CPU's power consumption scales as $P \propto C \cdot V^2 \cdot f$ where C is capacitance, V is voltage, and f is frequency. Reducing frequency by 2× reduces power by 2×, but reducing voltage by $\sqrt{2}$ reduces power by 4× (the dominant term is V^2). Modern CPUs actively vary both voltage and frequency.

P-states (Performance states): different (frequency, voltage) operating points. P0 is the highest (turbo boost), P1 is the base frequency, higher P-numbers are lower-power states. The OS/firmware selects P-states based on workload.

C-states (Idle states): power states when a core has no work to do. C0 = active, C1 = halt (clock gated), C3 = sleep (caches flushed), C6 = deep sleep (power gated, core state saved). Deeper C-states save more power but have longer **wake-up latency** (C6: ~200 µs to wake up).

C-state Wake Latency and Latency-Sensitive Applications

If a core is in C6 (deep sleep) and a network packet arrives, 200 µs pass before the core can process it. For trading systems and low-latency servers, C-states must be **disabled** (via BIOS or `/dev/cpu_dma_latency`) at the cost of higher idle power consumption. Many production latency-sensitive systems run at fixed P0 frequency (no DVFS) and disable all C-states.

1.22 GPU Architecture

GPU vs. CPU Design Philosophy

A CPU is optimised for **latency**: minimise the time to complete one task. It has few, very powerful cores with large caches, branch predictors, and OOO execution.

A GPU is optimised for **throughput**: maximise total work done per second. It has thousands of simpler cores (CUDA cores, shader processors) that execute many threads in lockstep, tolerating latency by switching threads when one stalls.

SIMT Execution Model

GPUs use **Single Instruction, Multiple Threads (SIMT)**: threads are grouped into **warps** (NVIDIA) or **wavefronts** (AMD) of 32–64 threads. All threads in a warp execute the **same instruction** simultaneously.
Thread divergence: if threads in a warp take different branches (`if/else`), the warp must execute *both* branches, disabling threads that didn't take each branch. Divergent branches halve throughput. Avoid branch divergence in GPU kernels by ensuring all threads in a warp take the same path.
Memory coalescing: if all threads in a warp access consecutive memory addresses, the accesses are coalesced into one wide memory transaction. Strided or random access by different threads generates many separate transactions — much slower.

GPU Memory Hierarchy

- **Registers**: fastest; per-thread. Spilling to local memory is slow.
- **Shared memory (L1)**: on-chip SRAM shared within a thread block. Low latency (∼4 cycles). Manually managed by the programmer. Key for tiled matrix multiply and other algorithms requiring data reuse.
- **L2 cache**: shared across the chip. Larger but slower.
- **Global memory (VRAM)**: the main GPU memory (HBM or GDDR). High bandwidth (∼1–3 TB/s for HBM) but high latency (∼200 cycles). The GPU hides this latency by switching to another warp.

1.23 Security: Spectre, Meltdown, and Speculative Execution Attacks

Meltdown (CVE-2017-5754)

Meltdown exploits the gap between speculative execution and privilege checks.
When user code accesses a kernel address, the CPU *speculatively* loads the data (before checking permissions), uses it in subsequent speculative instructions (leaving traces in the cache), then raises a fault. The permission check occurs *after* the speculative load.
Attack: After the fault is suppressed, the attacker measures which cache lines were loaded (via timing: **FLUSH+RELOAD**) — recovering the secret value that was in kernel memory. Any user process could read all kernel memory.
Fix: KPTI (Kernel Page Table Isolation) — separate page tables for kernel and user space. The kernel is not mapped in user space at all. Cost: every syscall/interrupt requires a TLB flush (expensive).

Spectre (CVE-2017-5753, CVE-2017-5715)

Spectre is more fundamental. It exploits the **branch predictor**:
1. The attacker **trains** the branch predictor by repeatedly running a branch with a particular outcome. 2. The victim code (in the same or a different process) has a bounds-checked array access: `if (x < array.len) use(array[x])`. 3. The attacker passes an out-of-bounds x. The branch predictor predicts "taken" (based on training). The CPU speculatively executes `use(array[x])` with the bad index, accessing secret memory and leaving cache traces. 4. The attacker reads the cache traces to recover the secret.
Unlike Meltdown, Spectre works *within* the victim's privilege level and is much harder to fix. Mitigations: **retpoline** (replace indirect branches to prevent speculative indirect calls), **IBPB/IBRS/STIBP** microcode updates, **site isolation** in browsers (no shared cache between origins).
Performance cost of mitigations: 5–30% for syscall-heavy workloads.

Chapter 1 Additional — Key Takeaways

- The **ROB** enables in-order commit with out-of-order execution. Register renaming eliminates false dependencies.
- **Store queues** and **load queues** manage speculative memory operations; store-to-load forwarding is a critical fast path.
- **Hardware prefetchers** detect access patterns; use `__builtin_prefetch` for irregular patterns like pointer chasing.
- **DRAM**: row buffer hits are fast; access memory sequentially.
- **C-states** save power but add wake latency; disable for latency-critical systems.
- **GPUs**: thousands of simple cores, SIMT execution, hide memory latency by switching warps. Avoid thread divergence and uncoalesced accesses.
- **Spectre/Meltdown**: speculative execution breaks security boundaries. Mitigations cost 5–30% performance on syscall-heavy workloads.

Chapter 2

Operating Systems

The operating system is the software layer between your program and the raw hardware. It manages who gets to run, how memory is shared, how files are stored, and how processes talk to devices. Understanding the OS turns mysterious behaviour into predictable mechanics.

2.1 What is an Operating System?

Operating System

An **operating system (OS)** is a program that:

- **Manages hardware resources** (CPU time, memory, devices) and shares them fairly among applications.
- **Provides abstractions** (files, processes, sockets) that hide messy hardware details from application programmers.
- **Enforces isolation and protection** — one buggy program can't corrupt another program's memory or crash the whole system.

Common OSes: Linux, Windows, macOS, Android, iOS.

The OS sits between the hardware and application programs. Applications talk to the OS through a well-defined interface called **system calls** (covered in §3). The OS, in turn, talks to hardware through **device drivers**.

Analogy: The Hotel Manager

Think of the OS as a hotel manager. Guests (processes) check in and get a room (memory). The manager ensures guests don't enter each other's rooms (memory isolation), schedules use of shared resources like the restaurant (CPU), and handles requests like "bring me a towel" (system calls) without the guest needing to know where the linen cupboard is (hardware details).

2.2 Kernel Space vs. User Space

Kernel and User Mode

Modern CPUs have at least two privilege levels:

- **Kernel mode (ring 0)** — full access to hardware, all instructions allowed, all memory accessible. The OS kernel runs here.
- **User mode (ring 3)** — restricted. Cannot directly access hardware, cannot execute privileged instructions, cannot touch other processes' memory. All user applications run here.

This separation prevents buggy or malicious user programs from corrupting hardware state or reading other processes' private data.

The **kernel** is the core of the OS that runs in kernel mode. It includes: the process scheduler, memory manager, file system drivers, network stack, and device drivers.

Why You Can't Just Read /dev/mem Directly

Reading raw hardware (physical memory, device registers) requires kernel mode privileges. If user programs could do this freely, any program could spy on passwords in another process's memory or corrupt the GPU state. The privilege separation enforces security and stability.

2.3 System Calls

System Call (syscall)

A **system call** is a controlled mechanism for user-mode code to request a service from the kernel. It involves:

1. User code puts the syscall number and arguments in registers.
2. A special CPU instruction (`SYSCALL` on x86-64) switches the CPU to kernel mode and jumps to the kernel's entry point.
3. The kernel validates arguments, performs the operation, and returns the result.
4. The CPU switches back to user mode.

Each transition costs ~50–300 ns. This is why tight loops shouldn't call `write()` per byte — batch I/O into large chunks.

Common System Calls

```
// File I/O syscalls
int fd = open("file.txt", O_RDONLY);   // open a file -> file descriptor
ssize_t n = read(fd, buf, 1024);        // read up to 1024 bytes
write(fd, buf, n);                      // write n bytes
close(fd);                              // release file descriptor

// Process syscalls
pid_t pid = fork();     // create a copy of current process
execv("/bin/ls", args);// replace current process with a new program
int status; waitpid(pid, &status, 0); // wait for child to exit

// Memory syscall
void *p = mmap(NULL, size, PROT_READ|PROT_WRITE,  // map memory
               MAP_PRIVATE|MAP_ANONYMOUS, -1, 0);
```

Libraries like the C standard library wrap syscalls in convenient functions. `printf()` internally calls `write()`. `malloc()` internally calls `brk()` or `mmap()`.

2.4 Processes

Process

A **process** is a running instance of a program. It consists of:

- **Code segment** — the compiled machine instructions (read-only).
- **Data segment** — global and static variables.
- **Heap** — dynamically allocated memory (`malloc`/`new`).
- **Stack** — function call frames, local variables.
- **Kernel metadata** — process ID (PID), open file descriptors, signal handlers, scheduling info. Stored in the **Process Control Block (PCB)**.

The OS gives each process its own **virtual address space** — an illusion of having the entire memory for itself. Virtual addresses are translated to physical RAM addresses by the hardware Memory Management Unit (MMU). This provides **isolation**: Process A at virtual address 0x1000 and Process B at virtual address 0x1000 refer to *completely different* physical memory locations.

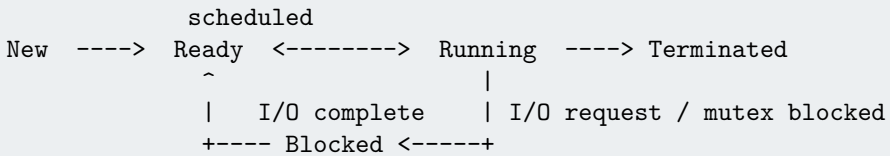
2.5 Process States and the Lifecycle

Process States

A process transitions through these states:

- **New** — being created (`fork()` called).
- **Ready** — waiting for CPU time; all other resources are available.
- **Running** — currently executing on a CPU core.
- **Blocked (Waiting)** — waiting for an event (I/O completion, mutex, timer). Cannot use the CPU productively.
- **Terminated (Zombie)** — execution finished; waiting for parent to read its exit status via `waitpid()`.

State Transitions



A process that terminates but whose parent hasn't called `waitpid()` becomes a **zombie** — it holds a PID and exit status entry, but no memory or CPU. Accumulating zombies wastes PID table entries. If the parent dies, `init` (PID 1) adopts the zombie and reaps it.

2.6 Threads vs. Processes

Thread

A **thread** is a unit of execution within a process. Multiple threads in the same process **share** the code, heap, global data, and open file descriptors. Each thread has its own **stack**, **registers**, and **program counter**. Creating a thread is much cheaper than forking a process because no memory is copied — threads share the same address space.

Property	Process	Thread
Address space	Own (isolated)	Shared within process
Creation cost	High (copy-on-write fork)	Low
Communication	IPC (pipes, sockets)	Shared memory
Crash isolation	Yes	No (one thread crashes all)
Typical use case	Separate programs	Parallel work within one program

POSIX Threads (pthreads) is the standard threading API on Linux/macOS. Java, Python, Go, Rust all provide higher-level threading abstractions built on top.

```c
#include <pthread.h>

void *worker(void *arg) {
    int *n = (int *)arg;
    printf("Thread working on: %d\n", *n);
    return NULL;
}

int main() {
    pthread_t t;
    int val = 42;
    pthread_create(&t, NULL, worker, &val);  // create thread
    pthread_join(t, NULL);                   // wait for thread to finish
    return 0;
}
```

2.7 Context Switching

Context Switch

When the OS switches which process/thread runs on a CPU core, it must:

1. **Save** the current thread's context: all registers, PC, stack pointer, into the thread's PCB/TCB.
2. **Select** the next thread to run (scheduler decision).
3. **Restore** the new thread's saved context into the CPU registers.
4. If switching between **processes** (not just threads), also switch the virtual address space (flush TLB, load new page table).

A thread-to-thread context switch within the same process costs $\sim$ 1–5 µs. A process-to-process context switch costs more: $\sim$ 5–20 µs.

Context switches are triggered by:

- A **timer interrupt** (the OS scheduler forcibly preempts a running thread after its time quantum expires — typically 1–10 ms).
- A **blocking syscall** (the thread blocks on I/O, so the OS runs something else).
- A **voluntary yield** (the thread calls `sched_yield()` or similar).

2.8 CPU Scheduling

The **scheduler** decides which ready thread runs next. Good scheduling aims to: maximise CPU utilisation, maximise throughput, minimise response time (for interactive tasks), and ensure fairness.

Key Scheduling Metrics

- **Turnaround time** — time from submission to completion.
- **Response time** — time from submission until first output.
- **Throughput** — jobs completed per unit time.
- **CPU utilisation** — fraction of time the CPU is doing useful work.
- **Fairness** — each process gets a fair share of CPU.

No single algorithm optimises all metrics simultaneously.

2.9 Scheduling Algorithms

First-Come, First-Served (FCFS): The simplest policy — run jobs in arrival order, non-preemptive.

The Convoy Effect

If a slow, CPU-bound process arrives first, all subsequent short processes wait behind it. Average wait time can be terrible. Not used in interactive systems.

Shortest Job First (SJF): Run the job with the shortest estimated burst time first. Optimal for minimising average turnaround. Problem: requires knowing burst time in advance (impractical). **SRTF** (Shortest Remaining Time First) is the preemptive variant.

Round Robin (RR):

Round Robin

Each process gets a fixed **time quantum** q (e.g., 10 ms). If it doesn't finish in q, it's preempted and put at the back of the ready queue. The next process runs for q, and so on.

- Good **response time** for interactive tasks.
- Small q: responsive but many context switches (overhead).
- Large q: fewer switches, degenerates toward FCFS.
- Typical Linux time quantum: 1–10 ms.

Priority Scheduling: Each process has a priority. The highest-priority ready process runs. Problem: **starvation** — low-priority processes may never run if high-priority ones keep arriving. Fix: **aging** — gradually increase priority of waiting processes.

Multi-Level Feedback Queue (MLFQ):

MLFQ — Used by Linux, macOS, Windows

Several queues with decreasing priority. New processes start at the top (highest priority, short quantum). If a process uses its full quantum repeatedly (CPU-bound), it moves down. I/O-bound processes that yield early stay near the top. Over time the system learns a process's behaviour and schedules appropriately. This achieves good response time for interactive tasks and good throughput for batch jobs simultaneously.

Linux uses the **Completely Fair Scheduler (CFS)**, which maintains a red-black tree ordered by *virtual runtime* — each process runs until its virtual runtime exceeds the minimum in the tree, keeping CPU time perfectly balanced.

2.10 Interrupts and Traps

Interrupt vs. Trap

- **Interrupt (hardware interrupt)** — an asynchronous signal from a device (e.g., NIC received a packet, keyboard key pressed, timer fired). The CPU pauses the current task, saves state, runs the **interrupt handler** (ISR), and resumes.
- **Trap (software interrupt / exception)** — synchronous, triggered by a CPU instruction. Examples: syscall instruction, division by zero, page fault, invalid instruction. The CPU immediately transfers control to the OS.

The timer interrupt is how preemptive scheduling works: the hardware timer fires every 1–10 ms, triggering an interrupt. The interrupt handler is the scheduler, which decides whether to preempt the running task or let it continue.

2.11 File Systems: Concepts

File System

A **file system** provides a hierarchical namespace (directories, files) and manages mapping between file names/offsets and physical storage blocks on disk.
A file is just a named sequence of bytes. The file system tracks:

- **Where** the data blocks are on disk.
- **Metadata**: permissions, timestamps, size, owner.
- **Directory structure**: name → file mapping.

Key file system operations map to syscalls: `open`, `read`, `write`, `seek`, `close`, `mkdir`, `unlink`, `rename`, `stat`.

File Descriptors: When you `open()` a file, the OS returns an integer **file descriptor** (fd). It's an index into the process's **open file table**. Standard file descriptors: 0 = stdin, 1 = stdout, 2 = stderr.

2.12 Inodes and Directory Structure

Inode (Index Node)

On Unix-like systems, every file and directory is represented by an **inode** — a data structure stored on disk containing:

- File type (regular, directory, symlink, device, ...)
- Permissions (rwxrwxrwx)
- Owner UID/GID
- Size in bytes
- Timestamps: access (atime), modification (mtime), change (ctime)
- Pointers to data blocks (direct, indirect, double-indirect)

Notably, the **file name is NOT in the inode**. Names live in directory entries.

A **directory** is a special file whose contents are a table of (name → inode number) mappings. When the OS resolves `/home/user/file.txt`, it:

1. Looks up `/` inode, finds its data blocks (the root directory table).
2. Finds "home" → inode #X in that table. Loads inode #X.
3. Finds "user" → inode #Y. Loads inode #Y.
4. Finds "file.txt" → inode #Z. Loads inode #Z to get file data.

Hard links: Multiple directory entries pointing to the same inode. Deleting one entry only deletes the file when the **link count** drops to zero.

Symbolic links (symlinks): A special file whose content is a path string. Following a symlink re-resolves the target path.

2.13 File System Implementations

FAT32 (File Allocation Table): Stores a table mapping each disk cluster to the next cluster in the file's chain. Simple, but no permissions, 4 GB file size limit, fragmentation prone. Used on USB drives for compatibility.

ext4 (Linux Default)

ext4 is a **journaling file system**. Before modifying metadata, it writes the intended change to a **journal** (a log). If the system crashes mid-write, the journal lets the OS replay or undo the incomplete operation at boot, ensuring consistency. Features: 16 TB max file size, extents (contiguous block ranges), delayed allocation, directory indexing. Default on most Linux distributions.

NTFS (Windows): Journaling, ACLs, compression, encryption, hard links.

Copy-on-Write (CoW) file systems (ZFS, Btrfs): Never overwrite data in-place. Old data is preserved until the new write is complete. Provides atomic snapshots and protects against partial writes corrupting data.

2.14 I/O Subsystem and Buffering

Raw disk I/O is slow. The OS uses several techniques to hide this latency:

Page Cache

The kernel maintains a **page cache** — a portion of RAM used to cache recently read/written disk data. When you `read()` a file, the OS first checks the page cache. On a hit, data is returned without any disk access. On Linux, the page cache can grow to use nearly all free RAM, and shrinks when applications need memory. `free` output showing "buff/cache" is this cache.

Buffered I/O vs. Direct I/O:

- **Buffered (default)**: Writes go to page cache; OS flushes to disk asynchronously (every few seconds or when `fsync()` is called). Fast for the application, but power loss before flush = data loss.
- **Direct I/O** (`O_DIRECT`): Bypasses page cache. Data goes straight from user buffer to disk. Used by databases that do their own caching.
- `fsync(fd)`: Force the OS to flush a file's dirty pages to disk. Databases call this on commit to guarantee durability.

2.15 Signals

Signals

A **signal** is an asynchronous notification sent to a process. When received, the process's normal execution is interrupted and a **signal handler** runs.
Common signals:

- `SIGINT` (2) — Ctrl+C; default action: terminate.
- `SIGTERM` (15) — polite termination request; can be caught and handled.
- `SIGKILL` (9) — unconditional kill; **cannot** be caught or ignored.
- `SIGSEGV` (11) — segmentation fault (invalid memory access).
- `SIGCHLD` — child process changed state (exited, stopped).
- `SIGALRM` — timer expired (`alarm()` syscall).

```c
#include <signal.h>
#include <stdio.h>

void handle_sigint(int sig) {
    printf("\nCaught Ctrl+C! Doing cleanup...\n");
    // do cleanup, then exit or set a flag
}

int main() {
    signal(SIGINT, handle_sigint);  // register handler
    while (1) { sleep(1); }         // run until Ctrl+C
    return 0;
}
```

Signal Handler Safety

Signal handlers run asynchronously, interrupting the program at any point. They should do minimal work: set a flag and return. Calling most library functions (like `printf()`, `malloc()`) from a signal handler is unsafe because those functions are not **async-signal-safe**. Only use functions listed in the POSIX async-signal-safe list inside signal handlers.

Chapter 2 — Key Takeaways

- The OS isolates processes via **kernel/user mode** and **virtual address spaces**.
- **System calls** are the only way for user programs to request kernel services; they have non-trivial overhead — batch I/O.
- **Threads** are lighter than processes; they share memory but crash together.
- **MLFQ** and **CFS** are the real-world scheduling algorithms; they favour I/O-bound (interactive) tasks.
- **Inodes** store file metadata; directories map names to inodes.
- The **page cache** makes reads fast; call `fsync()` to guarantee durability if you need it.
- **Signals** are asynchronous; signal handlers must be minimal and safe.

2.16 Inter-Process Communication (IPC)

Processes are isolated by design. When they need to communicate, the OS provides controlled channels.

Pipes

A **pipe** is a unidirectional, byte-stream channel with a kernel buffer (typically 64 KB on Linux). One end is the write end, the other is the read end.

```c
int fds[2];
pipe(fds);        // fds[0] = read end, fds[1] = write end
if (fork() == 0) {
    close(fds[0]);              // child: close read end
    write(fds[1], "hi", 2);    // child: write to pipe
    exit(0);
} else {
    close(fds[1]);              // parent: close write end
    char buf[10]; read(fds[0], buf, 10);  // parent: read from pipe
}
```

Named pipes (FIFOs) persist in the filesystem (`mkfifo()`) and allow unrelated processes to communicate. Throughput: ~3–10 GB/s for large writes.

Unix Domain Sockets

Full-duplex, bidirectional sockets within the same machine. Same API as TCP sockets (`socket()`, `connect()`, `send()`/`recv()`) but communicate via a filesystem path instead of IP:port. No network stack overhead. Throughput: ~5–15 GB/s. Latency: ~10 µs round-trip. Used by: Docker daemon (UNIX socket at `/var/run/docker.sock`), PostgreSQL, Redis (when connecting locally), systemd services.

Shared Memory

The fastest IPC: map the same physical pages into two processes' virtual address spaces. Writes by one process are immediately visible to the other with no kernel involvement (after setup).

```
// POSIX shared memory
int fd = shm_open("/myshm", O_CREAT|O_RDWR, 0666);
ftruncate(fd, 4096);
void *ptr = mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
// Now ptr points to shared memory visible to all processes that shm_open the same name
// MUST synchronise with semaphores or mutexes -- no automatic protection!
```

Throughput: memory speed (~50 GB/s). Used by: Chrome (renderer/browser process communication), high-frequency trading systems (kernel bypass + shared memory rings).

Message Queues (POSIX mq)

A kernel-maintained queue with typed messages. Supports priority. Processes `mq_send()` and `mq_receive()`. The kernel handles synchronisation. Easier to use correctly than shared memory, but slower (kernel involvement on every message). Rarely used in modern code; Unix domain sockets are generally preferred.

2.17 Copy-on-Write in fork()

Copy-on-Write (COW) fork

When `fork()` is called, it appears to create a complete copy of the parent's address space. Copying gigabytes of memory would be prohibitively expensive.
Instead, Linux uses **copy-on-write**: the parent's and child's page tables both point to the same physical pages, marked **read-only**.
When either process writes to a page, the MMU raises a **protection fault**. The kernel's fault handler:

1. Allocates a new physical page.
2. Copies the contents of the original page to the new page.
3. Updates the faulting process's page table to point to the new page (read/write).
4. Resumes execution.

Pages that are never written are never copied. A `fork()` followed immediately by `exec()` (common in shells) never copies any pages — `exec()` replaces the address space entirely.

Fork in Multi-Threaded Programs

`fork()` in a multi-threaded program creates a child with **only one thread** (the calling one). Any mutexes held by other threads at the time of fork are now permanently locked in the child — a deadlock waiting to happen. Only async-signal-safe functions should be called between `fork()` and `exec()` in the child. Prefer `posix_spawn()` or `vfork()` for launching child processes from multi-threaded parents.

2.18 Demand Paging and Page Faults

Demand Paging

When a process is loaded, its pages are **not** immediately loaded into RAM. The page table entries are marked "not present." On first access, a **page fault** occurs; the OS loads the page from disk (or the executable file), updates the page table, and resumes the faulting instruction.
Types of page faults:

- **Minor fault**: the page is in memory but not mapped (e.g., COW, first access to BSS). No disk I/O needed. Cost: ∼1 µs.
- **Major fault**: the page must be loaded from disk (executable, memory-mapped file, or swap). Cost: ∼1–10 ms (disk I/O).

`mlock()` pins pages in RAM, preventing them from being swapped. Used by databases, trading systems, and real-time applications to eliminate major faults.

2.19 cgroups and Linux Namespaces (The Foundation of Containers)

Linux Namespaces

A **namespace** wraps a global system resource in an abstraction so processes within the namespace see their own isolated instance. Linux provides 8 namespace types:

- **PID namespace**: each container sees its own PID 1. The init process of the container is PID 1 inside, but has a different PID from the host's view.
- **Network namespace**: each container has its own network stack (interfaces, routing table, iptables). Two containers can both bind port 80.
- **Mount namespace**: each container has its own filesystem view. `pivot_root()` or `chroot()` sets the root filesystem.
- **UTS namespace**: each container has its own hostname/domain name.
- **IPC namespace**: isolated System V IPC and POSIX message queues.
- **User namespace**: map UIDs/GIDs inside the namespace to different ones on the host (rootless containers).
- **Time namespace**: independent clock offsets (rare).
- **Cgroup namespace**: virtualised view of cgroup hierarchy.

`unshare(1)` creates new namespaces from the shell. `nsenter(1)` enters an existing namespace.

cgroups (Control Groups)

cgroups limit, account for, and isolate resource usage (CPU, memory, I/O, network) of groups of processes. The OS enforces these limits at the kernel level.
Key subsystems:

- **cpu**: limit CPU time (shares, quota, period). `cpu.cfs_quota_us=50000` on `cpu.cfs_period_us=100000` = 50% CPU limit.
- **memory**: set `memory.limit_in_bytes`. When the limit is exceeded, the OOM killer terminates processes in the cgroup.
- **blkio**: limit disk I/O bandwidth and IOPS per device.
- **net_cls/net_prio**: tag network packets for traffic shaping.

Docker containers = namespaces (isolation) + cgroups (resource limits) + overlayfs (layered filesystem).

cgroup Memory Limit in Docker

```
# Run container with 512MB memory limit and 1 CPU
docker run --memory=512m --cpus=1 my_image

# Under the hood, Docker writes to cgroup files:
# /sys/fs/cgroup/memory/docker/<id>/memory.limit_in_bytes = 536870912
# /sys/fs/cgroup/cpu/docker/<id>/cpu.cfs_quota_us = 100000
#   (with cfs_period_us = 100000 -> 1 full CPU)
```

2.20 epoll: Scalable I/O Event Notification

The Problem with select/poll

`select()` and `poll()` allow waiting for events on multiple file descriptors. But they have $O(n)$ overhead: you pass the entire fd set to the kernel on every call, and the kernel linearly scans it. For 10,000 connections, each `select()` call scans 10,000 fds even if only one is ready. This is why Apache (select-based) struggled above ~10,000 concurrent connections (the "C10K problem").

epoll — $O(1)$ Event Notification

epoll (Linux 2.6+) solves this:

- `epoll_create1()`: create an epoll instance (a kernel object maintaining a set of monitored fds).
- `epoll_ctl(epfd, EPOLL_CTL_ADD, fd, event)`: register interest in events on `fd`. This is done **once per fd**.
- `epoll_wait(epfd, events, maxevents, timeout)`: blocks until events occur. Returns **only the ready fds**, not the entire set. $O(1)$ per call regardless of how many fds are monitored (plus $O(k)$ for k ready events).

```c
int epfd = epoll_create1(0);
struct epoll_event ev = {.events = EPOLLIN, .data.fd = listen_fd};
epoll_ctl(epfd, EPOLL_CTL_ADD, listen_fd, &ev);

struct epoll_event events[64];
while (1) {
    int n = epoll_wait(epfd, events, 64, -1);  // block until ready
    for (int i = 0; i < n; i++) {
        if (events[i].data.fd == listen_fd) {
            int client = accept(listen_fd, NULL, NULL);
            ev.events = EPOLLIN | EPOLLET;   // edge-triggered
            ev.data.fd = client;
            epoll_ctl(epfd, EPOLL_CTL_ADD, client, &ev);
        } else {
            handle_client(events[i].data.fd);  // data ready to read
        }
    }
}
```

Edge-triggered (EPOLLET) vs. Level-triggered:

- **Level-triggered** (default): epoll notifies you as long as data is available. Easier to program correctly.
- **Edge-triggered**: notifies only when the state *changes* (new data arrives). You must read all available data in a loop. More efficient but requires non-blocking fds.

2.21 CFS: The Completely Fair Scheduler Internals

CFS Implementation

CFS maintains a **red-black tree** (ordered by **virtual runtime** — `vruntime`) of all runnable tasks. The leftmost node (lowest `vruntime`) is always the next task to run.
`vruntime` accumulates as a task runs, weighted by its **nice value**:

$$\Delta\text{vruntime} = \Delta\text{real\_time} \times \frac{\text{NICE\_0\_LOAD}}{\text{task\_weight}}$$

A higher-priority (lower nice) task has higher `task_weight`, so its `vruntime` grows more slowly — it accumulates virtual time slower than real time and gets more real CPU time.
When a task is preempted, it's re-inserted into the red-black tree. The task with the minimum `vruntime` runs next. All runnable tasks' `vruntime` values stay within one **target latency** (default: 6 ms for $\leq$8 tasks) of each other.

CFS Groups and Bandwidth Control

cgroup CPU bandwidth: CFS supports hard CPU limits via `cfs_quota_us`/`cfs_period_us`. A group is throttled once it consumes its quota in a period. Throttled tasks are re-queued after the period resets.
CPU affinity: `sched_setaffinity()` pins a process to specific CPU cores. Critical for latency-sensitive applications (avoids NUMA migrations and cache invalidation from cross-core migrations).

2.22 The Linux Network Stack

Network Stack Layers in the Kernel

- **NIC driver**: when a packet arrives, the NIC raises an interrupt. The driver uses **NAPI** (New API): instead of an interrupt per packet, the driver disables interrupts and polls for more packets (batching).
- **Softirq/ksoftirqd**: the driver enqueues packets to the per-CPU **RX ring buffer**. A software interrupt (`NET_RX_SOFTIRQ`) processes them out of interrupt context.
- **netfilter/iptables**: packets pass through hook points for filtering, NAT, connection tracking.
- **IP layer**: routing table lookup, fragmentation, TTL decrement.
- **TCP layer**: connection state machine, flow control, congestion control, reassembly, ACK generation.
- **Socket buffer (sk_buff)**: the kernel data structure carrying the packet through all layers. Zero-copy: data is passed by reference, not copied.
- **Socket receive buffer**: data placed here; `recv()` copies to user space.

Receive Livelock

Under extreme packet rates, interrupt handling can consume 100% CPU, leaving no time to process received packets. NAPI polling prevents this by batching. But with insufficient RX ring buffer (`ethtool -g eth0`), packets are dropped before NAPI can process them. Monitor with `ethtool -S eth0 | grep drop`.

Chapter 2 Additional — Key Takeaways

- **IPC mechanisms**: pipes (∼5 GB/s), UNIX sockets (∼10 GB/s), shared memory (memory speed). Choose based on topology and latency requirements.
- **COW fork**: no memory is copied until a page is written. Avoid writing large data structures after forking if you want speed.
- **Containers = namespaces + cgroups + overlayfs**. No kernel changes; just process isolation and resource accounting.
- **epoll** scales to millions of connections via $O(1)$ event delivery. Use edge-triggered mode for maximum efficiency.
- **CFS** uses a red-black tree keyed by virtual runtime. Nice values translate to CPU share weights.
- The Linux network stack is a layered pipeline of **sk_buffs**; NAPI polling prevents interrupt livelock at high packet rates.

Chapter 3

Memory Management & Allocation

Memory is the most finite resource a running program has. How the OS and runtime manage it determines correctness (no corruption), performance (no unnecessary latency), and scalability. This chapter covers the full stack: from virtual memory hardware all the way to garbage collectors.

3.1 Memory Layout of a Process

Process Memory Layout (Linux x86-64)

A process's virtual address space is divided into segments, from low to high address:

- **Text segment** — read-only machine code of the program.
- **Data segment** — initialised global/static variables (`int x = 5;`).
- **BSS segment** — zero-initialised globals (`int y;`). Not actually stored on disk; the OS zeros a page on first access.
- **Heap** — grows upward. Dynamic allocations (`malloc`/`new`).
- **Memory-mapped region** — shared libraries (`.so`/`.dll`), anonymous mappings (`mmap`).
- **Stack** — grows downward. Function call frames, local variables.
- **Kernel space** — upper portion reserved for the OS; not accessible from user mode.

Visualising the Address Space

```
High address  +----------------------+
              |    Kernel Space      |  (inaccessible from user mode)
              +----------------------+
              |       Stack          |  grows downward v
              |         v            |
              |    (unused gap)      |
              |         ^            |
              |   Memory-Mapped      |  shared libs, mmap()
              |       Heap           |  grows upward ^
              +----------------------+
              |        BSS           |
              |       Data           |
              |       Text           |  read-only code
Low address   +----------------------+
```

3.2 Stack vs. Heap

Stack Memory

The **stack** is automatically managed. When a function is called, a **stack frame** is pushed: it contains local variables, saved registers, and the return address. When the function returns, the frame is popped in $O(1)$.
Characteristics:

- Allocation: essentially free (just decrement the stack pointer).
- Size: limited; typically 1–8 MB per thread on Linux (configurable via `ulimit`).
- Lifetime: tied to the function call. Variables disappear on return.
- Access: LIFO (Last In, First Out).

Exceeding the stack limit causes a **stack overflow** (SIGSEGV).

Heap Memory

The **heap** is managed explicitly (C/C++) or by a garbage collector (Java, Python, Go). Used for objects with lifetimes not tied to a function.
Characteristics:

- Allocation: slower (must find a free block, update bookkeeping).
- Size: can grow up to available virtual memory (terabytes on 64-bit systems).
- Lifetime: until explicitly freed (`free()`/`delete`) or GC'd.
- Access: random; any pointer can reference any live heap object.

```
void example() {
    int local = 42;         // stack: auto-freed when function returns
    int *heap = malloc(sizeof(int) * 100);  // heap: lives until free()
    heap[0] = 1;
    free(heap);             // must free manually in C; else memory leak
}
```

3.3 Virtual Memory

Virtual Memory

Virtual memory gives each process the illusion of having a large, private, contiguous address space — even if physical RAM is fragmented or smaller.
The CPU's **Memory Management Unit (MMU)** translates every memory access:

$$\text{Virtual Address} \xrightarrow{\text{MMU}} \text{Physical Address}$$

This translation happens automatically in hardware with no software overhead per access.
Benefits:

- **Isolation** — each process's virtual addresses map to different physical addresses.
- **Overcommitment** — processes can allocate more virtual memory than physical RAM exists; unused pages are never loaded.
- **Shared memory** — multiple processes can map the same physical page (e.g., a shared library loaded once into RAM, mapped into every process).
- **Swapping** — infrequently used pages can be evicted to disk (swap space) to free RAM.

3.4 Paging

Paging

Virtual and physical memory are divided into fixed-size chunks called **pages** (virtual) and **frames** (physical). Typical page size: **4 KB**.
The OS maintains a **page table** for each process — an array mapping virtual page numbers to physical frame numbers. The MMU uses this table for every memory access.
If a virtual page has no corresponding frame (not yet loaded, or swapped out), accessing it triggers a **page fault**: the OS loads the page from disk/backing store into a free frame, updates the page table, and restarts the instruction.

Huge Pages

For programs with very large working sets (databases, in-memory caches), 4 KB pages mean enormous page tables and many TLB misses. **Huge pages** (2 MB or 1 GB on x86-64) reduce TLB pressure drastically. Linux supports huge pages via `mmap()` with `MAP_HUGETLB`, or Transparent Huge Pages (THP) automatically.

3.5 Page Tables and the TLB

Multi-Level Page Tables: A 64-bit address space with 4 KB pages would need a page table with 2^{52} entries — trillions of entries. Instead, x86-64 uses a **4-level page table**: the virtual address is split into four 9-bit indices and a 12-bit page offset. Only the portions of the page table that are actually used need to be allocated.

TLB — Translation Lookaside Buffer

Walking the 4-level page table on every memory access would be extremely slow (4 extra memory accesses per load/store). The **TLB** is a small, fast, fully-associative cache of recent virtual→physical translations.

- L1 TLB: ~64 entries, 0 extra cycles on hit.
- L2 TLB: ~1024 entries, 5–7 extra cycles on hit.
- **TLB miss**: MMU must walk the page table in memory: ~10–200 cycles.

On a context switch between processes, the TLB must be flushed (TLB shootdown) since mappings differ. This is part of why process switches are more expensive than thread switches.

3.6 Segmentation

Older architectures (and still present in x86 in legacy form) used **segmentation**: the address space is divided into variable-size **segments** (code, data, stack), each with a base address and a limit.

Modern 64-bit OSes (Linux, Windows in 64-bit mode) use a **flat address space**: segments are set to cover the full 64-bit range, and all address translation is done via paging. Segmentation still exists at the hardware level but is effectively disabled.

Why segmentation was useful: natural separation of code, data, and stack with hardware-enforced bounds. If code tries to write to the code segment, the hardware raises a protection fault. Paging achieves similar protection with finer granularity.

3.7 Page Replacement Algorithms

When physical memory is full and a new page must be brought in, the OS must evict an existing page. Choosing the right page to evict is critical for performance.

Optimal (OPT)

Evict the page that will not be used for the longest time in the future. Provably optimal (fewest page faults), but requires future knowledge — impossible to implement in practice. Used as a **benchmark** to evaluate other algorithms.

LRU — Least Recently Used

Evict the page that has not been accessed for the longest time. **Rationale:** temporal locality — recently used pages are likely needed again.

Challenge: True LRU requires updating a timestamp on every memory access — too expensive. Real systems use **approximations**:

- **Clock algorithm** (Second Chance): pages in a circular list with a reference bit. On fault, the "hand" scans; if reference bit = 1, clear it and advance; if 0, evict. A full rotation gives pages a "second chance."
- Linux uses a two-list approach: active and inactive lists with LRU order.

FIFO — First In, First Out

Evict the oldest-loaded page. Simple but can evict heavily-used pages. **Bélády's anomaly:** adding more physical frames can increase page faults with FIFO (counterintuitive, and unique to FIFO among common algorithms).

3.8 Dynamic Memory Allocation: malloc Internals

How malloc Works

`malloc(n)` must find a contiguous free block of at least n bytes on the heap. The allocator maintains a data structure tracking free blocks. Common strategies:

- **First-fit**: scan from the start, return the first block $\geq n$. Fast to allocate, but fragmentation builds up at the front.
- **Best-fit**: find the smallest block $\geq n$. Minimises wasted space but can create many tiny unusable fragments.
- **Segregated free lists**: maintain separate free lists for different size classes (e.g., 8, 16, 32, 64, ... bytes). Fast $O(1)$ allocation for common sizes.

When the heap runs out of space, the allocator calls `sbrk()` or `mmap()` to request more pages from the OS.

Memory Fragmentation

After many alloc/free cycles, free memory may exist only in small, scattered chunks — even though the total free bytes are sufficient. This is **external fragmentation**. **Internal fragmentation** is wasted space within an allocated block (e.g., allocating 32 bytes when 17 were requested, because the allocator rounds up to the next size class).

3.9 The Buddy System

Buddy System Allocator

Memory is divided into blocks of power-of-two sizes. To allocate n bytes:

1. Round n up to the nearest power of two: 2^k.
2. Find a free block of size 2^k. If none exists, split a 2^{k+1} block into two **buddies** of size 2^k. Recurse upward if needed.
3. Return one buddy; keep the other in the free list for 2^k.

To free: check if the freed block's **buddy** is also free. If so, merge them back into a 2^{k+1} block. Repeat upward (**coalescing**).

Pros: Simple, fast coalescing (buddy address is easily computed via XOR).
Cons: Internal fragmentation (up to 50% waste), only power-of-two sizes.

The Linux kernel uses the buddy system for page-level allocation.

3.10 Slab Allocation

Slab Allocator

Used by the Linux kernel for kernel objects (e.g., `struct task_struct`, `struct inode`). The idea:

1. Pre-allocate **slabs** — chunks of pages, each divided into fixed-size **slots** for one specific object type.
2. Allocation: pop a free slot from the appropriate slab — $O(1)$.
3. Deallocation: push the slot back — $O(1)$.
4. Object **constructor/destructor** can be called on allocation/free, reusing initialised state (caching warm objects).

Benefits: eliminates fragmentation for fixed-size objects, extremely fast, no need to zero-initialize on each allocation (reuse warmed objects), cache-line aligned.

The **jemalloc** and **tcmalloc** allocators (used by Firefox, Chrome, and most servers) use a similar segregated-slab approach for user-space allocation, with per-thread caches to reduce lock contention.

3.11 Garbage Collection: Overview

In languages like Java, Python, Go, JavaScript, and C#, the runtime automatically reclaims memory — you don't call `free()`.

Garbage Collection (GC)

A **garbage collector** automatically identifies and reclaims memory that is no longer reachable by any live reference in the program.
Key challenge: distinguishing **live** objects (still referenced) from **garbage** (unreachable). GC does this without programmer intervention.
GC trade-offs:

- **Throughput**: time spent in GC vs. running application code.
- **Pause time**: GC may stop the world (STW) — freeze all threads while it runs. Latency-sensitive apps need short, predictable pauses.
- **Footprint**: extra memory needed by GC metadata and headroom.

3.12 Reference Counting

Reference Counting

Every object carries a **reference count** — the number of pointers that refer to it. When a reference is created, increment. When a reference is destroyed, decrement. When the count reaches **zero**, the object is garbage and is freed immediately.
Used by: CPython, Rust's `Rc`/`Arc`, Swift's ARC, COM.

The Cycle Problem

Reference counting **cannot collect cycles**: if object A refers to B and B refers to A, both have count ≥ 1 even if nothing else references them — they're leaked forever.
CPython solves this with a supplemental **cyclic GC** that periodically identifies reference cycles. Rust `Rc` simply does not handle cycles (use `Weak` pointers to break them manually).

Pros: Immediate reclamation, no pause, low overhead for long-lived objects.
Cons: Per-reference-operation overhead, can't collect cycles, cache ping-pong on the reference count in multi-threaded code (`Arc` increments require atomics).

3.13 Mark-and-Sweep GC

Mark-and-Sweep

1. **Mark phase**: start from **GC roots** (stack variables, global variables, registers). Traverse all reachable objects via references, marking each visited object.
2. **Sweep phase**: scan the entire heap. Any unmarked object is garbage; reclaim its memory.

All live objects are guaranteed to be found, including cycles.

Stop-the-World (STW): The simplest implementation pauses all application threads during both phases. For large heaps, this pause can be seconds — unacceptable for interactive or low-latency applications.

Incremental and Concurrent GC: Modern collectors (JVM G1, Go's GC) do most of the marking concurrently with application threads, only stopping briefly for synchronisation. Go targets sub-millisecond pauses.

3.14 Generational Garbage Collection

The Generational Hypothesis

Empirically, **most objects die young** — a temporary string, a loop iteration object, a short-lived request object. Objects that survive one GC cycle are likely to survive many more. This observation motivates generational GC.

Generational GC

The heap is divided into **generations**:

- **Young generation** (Eden + Survivor spaces): New allocations go here. Small, collected frequently (**minor GC**). Most objects die here. Minor GC is fast because it only scans the young generation.
- **Old generation (Tenured)**: Objects that survive N minor GCs are **promoted** here. Collected infrequently (**major GC**).

The JVM's default GC (G1) divides the heap into regions, predicts collection times, and meets soft pause-time goals. The JVM's ZGC targets <1 ms pauses regardless of heap size using concurrent relocation.

Copying collection (used for young gen): Divide the young gen into two halves. Allocate in one; on GC, copy all live objects to the other half. All garbage is left behind. Compacts memory for free — no fragmentation, fast allocation (just bump a pointer).

3.15 Memory-Mapped Files

mmap()

`mmap()` maps a file (or anonymous memory) directly into the process's virtual address space. Reads and writes to the mapped region automatically read/write the file via the page cache.

Benefits:

- No explicit `read()`/`write()` syscalls — just pointer accesses.
- Pages are demand-loaded (only the accessed portion is read from disk).
- Multiple processes can map the same file, sharing one physical copy.
- Used by: dynamic linker (loading `.so` libraries), databases (LMDB, SQLite WAL), large file processing.

```c
#include <sys/mman.h>
#include <fcntl.h>

int fd = open("data.bin", O_RDONLY);
struct stat sb; fstat(fd, &sb);
// Map entire file into memory
char *data = mmap(NULL, sb.st_size, PROT_READ, MAP_SHARED, fd, 0);
// Access file contents as if it were an array
printf("First byte: %c\n", data[0]);
munmap(data, sb.st_size);   // unmap when done
close(fd);
```

3.16 NUMA — Non-Uniform Memory Access

NUMA

Multi-socket server systems have multiple CPU packages, each with its own local RAM. Accessing **local RAM** (same socket) is fast (~100 ns). Accessing **remote RAM** (different socket, via QPI/UPI interconnect) is 1.5–3× slower. This is **Non-Uniform Memory Access (NUMA)**.

The OS is NUMA-aware: it tries to allocate memory on the same NUMA node as the CPU that requested it (**NUMA-local allocation**). Linux provides the `numactl` tool and `mbind()` syscall for manual NUMA control.

NUMA Best Practices

- Pin threads to cores on one NUMA node (`numactl -cpunodebind=0`).
- Allocate memory on the same node (`numactl -membind=0`).
- Avoid sharing data between threads on different NUMA nodes (all remote memory accesses and cache coherence traffic crosses the slow interconnect).
- Monitor with `numastat` to detect remote memory accesses.

Chapter 3 — Key Takeaways

- The **stack** is fast and automatic; the **heap** is flexible but needs explicit management (or GC).
- **Virtual memory** gives each process an isolated, large address space; the **MMU + page tables** do the translation.
- The **TLB** caches page table entries; TLB misses are expensive. Large pages reduce TLB pressure.
- Page replacement algorithms (**LRU/Clock**) evict cold pages to disk.
- `malloc` uses **segregated free lists**; the kernel uses **buddy system + slab** allocation.
- **Reference counting** is simple but leaks cycles. **Mark-and-sweep** handles cycles but can pause.
- **Generational GC** exploits the "most objects die young" hypothesis for efficient collection.
- On multi-socket systems, **NUMA awareness** is essential for performance.

3.17 Buffer Overflows, Stack Canaries, and ASLR

Stack Buffer Overflow

A **buffer overflow** occurs when writing past the end of a stack-allocated buffer overwrites adjacent stack data — including the **return address**.

```
void vulnerable(const char *input) {
    char buf[64];
    strcpy(buf, input);  // no bounds check: if len(input) > 64, overflow!
    // If input is 72 bytes, we overwrite 8 bytes past buf
    // On a 64-bit system: buf[64..71] = saved rbp, buf[72..79] = return address
}
```

An attacker crafts `input` to overwrite the return address with the address of their shellcode (or a ROP gadget), taking control of the program.

Stack Canaries

The compiler (`-fstack-protector`) inserts a random **canary value** on the stack between local variables and the saved return address. Before returning, the function checks if the canary was modified. If so, the program immediately aborts.

```
// Compiler-generated (simplified):
void vulnerable(const char *input) {
    unsigned long canary = __stack_chk_guard;  // random value per-process
    char buf[64];
    strcpy(buf, input);
    if (canary != __stack_chk_guard) __stack_chk_fail();  // abort!
}
```

Canaries don't prevent overflows; they detect them before the return. Bypassed by: format string attacks (can leak the canary value) or overwriting data before the canary.

ASLR — Address Space Layout Randomization

ASLR randomises the base addresses of the stack, heap, and shared libraries at each program invocation. An attacker who doesn't know where the stack/heap is can't hardcode the address to jump to.

- `/proc/sys/kernel/randomize_va_space`: 0=off, 1=partial, 2=full.
- **PIE** (Position Independent Executable): the executable itself is also randomly placed. Required for full ASLR. Compile with `-fPIE -pie`.
- Entropy: 28 bits of stack ASLR on 64-bit Linux = 2^{28} possible positions. Brute-forcing is impractical remotely but possible locally via fork-based attacks (child inherits parent's address layout).

ASLR + canaries + NX (non-executable stack) + PIE form the core of modern stack protection. Bypassed via: information leak vulnerabilities, heap spraying, ROP chains.

3.18 DMA — Direct Memory Access

DMA

Without DMA, the CPU must copy every byte between I/O devices and memory — wasting CPU cycles on memory-copy loops. **DMA** allows devices to access main memory directly without CPU involvement.

Operation:

1. CPU programs the DMA controller with: source address, destination address, byte count.
2. DMA controller transfers data independently, using memory bus cycles.
3. When complete, DMA controller raises an interrupt to notify the CPU.

The CPU is free to do other work during the transfer.

IOMMU and DMA Security

A malicious or buggy DMA-capable device could read/write any physical memory address. The **IOMMU** (I/O Memory Management Unit) applies address translation and access control to DMA operations: devices can only access memory regions the OS has explicitly granted. Essential for **PCIe device passthrough** to VMs (the VM's device driver directly controls the physical device without compromising the host).

Linux: `iommu=on` kernel parameter, `vfio` driver for VM passthrough.

3.19 Memory Debugging with Valgrind and AddressSanitizer

Valgrind (memcheck)

Valgrind is a dynamic binary analysis tool. The **memcheck** tool detects:

- **Use after free**: accessing memory after calling `free()`.
- **Heap buffer overflows**: writing past the end of an allocation.
- **Uninitialized reads**: using a variable before assigning it.
- **Memory leaks**: allocated memory never freed.
- **Double free**: calling `free()` twice on the same pointer.

Overhead: **10–50x slower** than normal execution. Not suitable for production. Used in test/debug builds.

AddressSanitizer (ASan)

A compiler-based tool (`-fsanitize=address`) with ~**2x** overhead (much faster than Valgrind). ASan inserts **shadow memory** tracking which bytes are addressable. On every memory access, the shadow is checked.

```
# Compile with ASan
g++ -fsanitize=address -g -O1 program.cpp -o program
./program
# On error:
# ==1234==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x...
# READ of size 4 at 0x... thread T0
#     #0 main program.cpp:10
```

LeakSanitizer (LSan): detects leaks at program exit (built into ASan).
MemorySanitizer (MSan): detects uninitialized reads.
ThreadSanitizer (TSan): detects data races.
UBSan: detects undefined behavior (integer overflow, null dereference, etc.).

3.20 Rust's Ownership Model vs. Garbage Collection

Rust Ownership

Rust achieves memory safety *without* a garbage collector using compile-time ownership rules:

- **Ownership**: each value has exactly one owner. When the owner goes out of scope, the value is dropped (destructor called, memory freed).
- **Borrowing**: you can have either one mutable reference (`&mut T`) or any number of immutable references (`&T`) to a value at any time. Not both. Enforced at compile time.
- **Lifetimes**: references cannot outlive the data they point to. The borrow checker verifies this at compile time.

Result: no dangling pointers, no use-after-free, no data races — all guaranteed at compile time with zero runtime overhead.

```
    let s = String::from("hello");  // s owns the String
    let r1 = &s;                    // immutable borrow -- OK
    let r2 = &s;                    // second immutable borrow -- OK
    // let r3 = &mut s;             // ERROR: can't borrow mutably while immutably borrowed
    println!("{} {}", r1, r2);
}   // s goes out of scope -> String::drop() called -> memory freed automatically
    // No GC pause, no leak, no dangling reference possible.
```

When to Choose Rust vs. GC'd Language

- **Use GC** (Java, Go, Python): when developer productivity is paramount, garbage pauses are acceptable, and the working set fits comfortably in memory.
- **Use Rust**: when you need predictable latency (no GC pauses), minimal memory overhead (no GC metadata), maximum control over allocation, or when interfacing closely with hardware/OS. Ideal for: systems software, game engines, network proxies (Linkerd2 proxy), WebAssembly runtimes (Wasmtime), operating systems (Linux kernel drivers are being written in Rust).

3.21 Weak References and Object Graphs

Weak References

A **weak reference** is a reference to an object that **does not prevent garbage collection**. The GC may collect the object; accessing a weak reference after collection returns null/None.

Use cases:

- **Caches**: cache an object weakly so it can be collected if memory is needed. On next access, recompute.
- **Breaking reference cycles**: if A holds a strong ref to B and B holds a weak ref to A, the cycle is broken (B doesn't prevent A's collection).
- **Observer pattern**: observers registered with weak references are auto-removed when they're collected. No explicit deregistration needed.

Java: `WeakReference<T>`. Python: `weakref.ref(obj)`. Rust: `Weak<T>` (for `Rc`).

Chapter 3 Additional — Key Takeaways

- **Buffer overflows** overwrite the return address. Mitigations: canaries, ASLR/PIE, NX stack, and bounds-safe languages.
- **ASLR** randomises address layout; PIE randomises the executable itself.
- **DMA** lets devices access RAM without CPU; **IOMMU** constrains which addresses devices can reach.
- **ASan** (2x overhead) is vastly more practical than Valgrind (50x) for catching memory bugs during development.
- **Rust**'s ownership model gives memory safety at zero runtime cost — no GC, no pauses, no dangling pointers.
- **Weak references** break reference cycles and enable safe caches.

Chapter 4

Concurrency & Parallelism

Concurrency is one of the hardest topics in systems programming. The problems are subtle, the bugs are non-deterministic, and the solutions require understanding both the hardware and the programming model. Get this chapter solid and you'll be ahead of most engineers.

4.1 Concurrency vs. Parallelism

Definitions

- **Concurrency** — structuring a program as multiple tasks that can be *in progress* at the same time. They may not actually run simultaneously — a single-core CPU interleaves them via context switching. Concurrency is about **structure**.
- **Parallelism** — multiple tasks *actually executing simultaneously* on multiple CPU cores. Parallelism is about **execution**.

A program can be concurrent without being parallel (single-core, multithreaded), or parallel without being concurrent (SIMD: one instruction, multiple data).

Analogy: The Kitchen

Concurrency: One chef juggling multiple dishes — while the pasta boils, they chop vegetables; while the oven preheats, they prepare the sauce. One person, many tasks in progress.
Parallelism: Multiple chefs, each working on a different dish simultaneously. Many people, multiple tasks at once.

4.2 Threads and the Threading Model

Each **thread** of execution has its own:

- Program counter (PC) and register file
- Stack (local variables, call frames)

And shares with other threads in the same process:

- Heap and global variables
- Open file descriptors
- Signal handlers
- Code (text segment)

User-level threads vs. kernel threads:

- **Kernel threads**: the OS knows about them; the OS schedules them. A blocking syscall on one thread doesn't block others. Linux uses 1:1 mapping (each user thread = one kernel thread). `pthread_create` creates kernel threads.

- **User-level threads (green threads / goroutines / fibers)**: the runtime manages scheduling in user space. Extremely cheap to create (Go goroutines: ∼4 KB stack vs. ∼8 MB for kernel threads). The runtime multiplexes M goroutines onto N OS threads (M:N scheduling).

4.3 Race Conditions

Race Condition

A **race condition** occurs when the correctness of a program depends on the relative timing or interleaving of multiple threads, and at least one thread **modifies shared state** without proper synchronisation.
The result is non-deterministic — the program may produce different outputs on each run, making bugs extremely difficult to reproduce and debug.

Classic Race: Counter Increment

```
int counter = 0;   // shared global

// Thread 1:              // Thread 2:
counter++;                counter++;

// Compiles to 3 instructions each:
// LOAD counter into register
// ADD 1 to register
// STORE register back to counter

// With unlucky interleaving:
// T1: LOAD counter (reads 0)
// T2: LOAD counter (reads 0)
// T1: ADD -> 1, STORE -> counter = 1
// T2: ADD -> 1, STORE -> counter = 1  <- Lost update! Should be 2.
```

After 1,000,000 increments from two threads, the result may be anywhere from 1,000,000 to 2,000,000 depending on interleaving.

4.4 Critical Sections and Mutual Exclusion

Critical Section

A **critical section** is a piece of code that accesses shared state and must not be executed by more than one thread at a time.
Requirements for correct mutual exclusion:

1. **Mutual exclusion** — at most one thread in the critical section.
2. **Progress** — if no thread is in the critical section, a waiting thread must eventually enter.
3. **Bounded waiting** — a thread must eventually get in (no starvation).

4.5 Mutexes

Mutex (Mutual Exclusion Lock)

A **mutex** is a synchronisation primitive with two operations:

- **lock()** (or `acquire()`): if the mutex is free, take it and enter the critical section. If it's held by another thread, **block** (sleep) until it's released.
- **unlock()** (or `release()`): release the mutex, unblocking one waiting thread.

A mutex provides **ownership**: only the thread that locked it can unlock it.

```
#include <pthread.h>
pthread_mutex_t mu = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&mu);    // enter critical section
        counter++;                  // safe: only one thread here
        pthread_mutex_unlock(&mu);  // leave critical section
```

```
    }
    return NULL;
}
```

Lock Granularity

Coarse-grained locking (one lock for everything): simple, but threads serialise unnecessarily. **Fine-grained locking** (one lock per data structure or hash bucket): more concurrency, but complex, harder to reason about, and deadlock-prone.
Always start with the simplest correct locking and optimise only when profiling shows lock contention is the bottleneck.

4.6 Spinlocks

Spinlock

Instead of sleeping when the lock is held, a **spinlock** repeatedly checks (spins) in a loop until the lock becomes available:

```
// Conceptual spinlock
while (lock.test_and_set()) { /* spin */ }
// ... critical section ...
lock.clear();
```

When to use: Only when the critical section is extremely short (a few nanoseconds) and the CPU that holds the lock is running right now. In that case, spinning for a few cycles beats the ~ 5 µs overhead of a sleep/wake cycle.
Never use on a single-core CPU: spinning blocks the only core, so the lock-holder can never run to release it — deadlock.

4.7 Semaphores

Semaphore

A **semaphore** is a generalised synchronisation primitive with an integer count and two atomic operations:
- **wait() / P() / down()**: decrement the count. If the result is negative, the calling thread blocks.
- **signal() / V() / up()**: increment the count. If any threads are blocked, wake one.

A semaphore initialised to **1** behaves like a mutex (binary semaphore). A semaphore initialised to **N** allows up to N threads through simultaneously (counting semaphore) — useful for rate limiting or bounded buffers.

Semaphore for Resource Pooling

```
// Allow at most 10 concurrent DB connections
sem_t db_sem;
sem_init(&db_sem, 0, 10);    // count = 10

void use_database() {
    sem_wait(&db_sem);       // blocks if 10 connections already active
    // ... use database connection ...
    sem_post(&db_sem);       // release slot for another thread
}
```

4.8 Monitors and Condition Variables

Condition Variable

A **condition variable** (CV) lets threads wait for a condition to become true, releasing the mutex while waiting (so other threads can make progress).
Three operations (always used with a mutex):

- `wait(mu, cv)`: atomically release `mu` and sleep on `cv`.
- `signal(cv)` / `notify_one()`: wake one thread waiting on `cv`.
- `broadcast(cv)` / `notify_all()`: wake all threads waiting on `cv`.

After being woken, the thread **re-acquires the mutex** and re-checks the condition (use a `while` loop, not `if` — spurious wakeups are allowed by POSIX).

```
pthread_mutex_t mu = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
bool data_ready = false;

// Consumer thread
void *consumer(void *_) {
    pthread_mutex_lock(&mu);
    while (!data_ready)                 // while, not if!
        pthread_cond_wait(&cv, &mu);    // release mu, sleep; re-acquire on wake
    // process data...
    pthread_mutex_unlock(&mu);
    return NULL;
}

// Producer thread
void *producer(void *_) {
    // ... produce data ...
    pthread_mutex_lock(&mu);
    data_ready = true;
    pthread_cond_signal(&cv);           // wake consumer
    pthread_mutex_unlock(&mu);
    return NULL;
}
```

4.9 The Producer–Consumer Problem

Producer-Consumer (Bounded Buffer)

A classic concurrency problem: producers generate data and place it in a bounded buffer; consumers take data out. Constraints:

- Producers must wait when the buffer is **full**.
- Consumers must wait when the buffer is **empty**.
- Only one thread accesses the buffer at a time (mutual exclusion).

```
queue<int> buf;           // shared buffer
const int CAPACITY = 10;
mutex mu; condition_variable not_full, not_empty;

void producer(int item) {
    unique_lock<mutex> lk(mu);
    not_full.wait(lk, [&]{ return buf.size() < CAPACITY; });
    buf.push(item);
    not_empty.notify_one();
}

int consumer() {
    unique_lock<mutex> lk(mu);
    not_empty.wait(lk, [&]{ return !buf.empty(); });
    int item = buf.front(); buf.pop();
    not_full.notify_one();
    return item;
}
```

4.10 The Reader–Writer Problem

Reader-Writer Problem

Multiple readers can access shared data simultaneously (reads don't interfere with each other). Writers need exclusive access (a write must block all other readers and writers).
Reader-preference: readers are never blocked unless a writer holds the lock. Writers can starve.
Writer-preference: once a writer is waiting, no new readers are admitted. Prevents writer starvation.
Fair: readers and writers alternate reasonably; neither starves.

C++ `std::shared_mutex` (C++17) implements a reader-writer lock:

```
#include <shared_mutex>
std::shared_mutex rw_lock;

void reader() {
    std::shared_lock<std::shared_mutex> lk(rw_lock);  // shared (read) lock
    // multiple readers can hold shared_lock simultaneously
}
void writer() {
    std::unique_lock<std::shared_mutex> lk(rw_lock);  // exclusive (write) lock
    // only one writer, no concurrent readers
}
```

4.11 Deadlock

Deadlock

A **deadlock** occurs when a set of threads are each waiting for a resource held by another thread in the set — a circular wait with no thread able to proceed.
Coffman's four necessary conditions (all must hold for deadlock):

1. **Mutual exclusion** — resources cannot be shared.
2. **Hold and wait** — a thread holds a resource while waiting for another.
3. **No preemption** — resources cannot be forcibly taken from a thread.
4. **Circular wait** — a cycle of threads each waiting for the next.

Eliminate **any one** condition and deadlock cannot occur.

Classic Deadlock

```
// Thread 1:                    // Thread 2:
lock(mutex_A);                  lock(mutex_B);
// ... work ...                 // ... work ...
lock(mutex_B);  // waits for B   lock(mutex_A);  // waits for A
// DEADLOCK: T1 holds A waiting for B; T2 holds B waiting for A
```

Preventing Deadlock

- **Lock ordering**: always acquire locks in the same global order (e.g., always lock mutex_A before mutex_B). Eliminates circular wait.
- **Try-lock with backoff**: use `try_lock()`; if it fails, release all held locks and retry after a random delay.
- **Lock-free algorithms**: eliminate locks entirely.
- **Single lock**: if possible, use one coarse-grained lock. No circular wait possible.

4.12 Lock-Free Programming and Atomic Operations

Atomic Operation

An **atomic operation** is one that appears instantaneous to all other threads — it cannot be interrupted mid-way. Modern CPUs provide atomic instructions:

- `fetch_add`: atomically add and return old value.
- `compare_and_swap (CAS)`: atomically: if `*ptr == expected`, set `*ptr = desired` and return true; else return false.
- `test_and_set`, `exchange`, `fetch_or`, etc.

These are implemented as single hardware instructions (e.g., `LOCK CMPXCHG` on x86) that the CPU guarantees are atomic across all cores.

Lock-Free Counter with fetch_add

```
#include <atomic>
std::atomic<int> counter{0};

void increment() {
    counter.fetch_add(1, std::memory_order_relaxed);  // atomic, no lock needed
}
// This is how all standard atomic counters (metrics, statistics) should be done.
```

Compare-And-Swap (CAS) Loop

The fundamental building block of lock-free data structures:

```
// Lock-free stack push
struct Node { int val; Node *next; };
std::atomic<Node*> head{nullptr};

void push(int val) {
    Node *node = new Node{val, nullptr};
    do {
        node->next = head.load();                // read current head
    } while (!head.compare_exchange_weak(        // CAS: if head == node->next,
                node->next, node));              //      set head = node; else retry
}
```

If two threads try to push simultaneously, only one CAS succeeds; the other retries with the updated head. No lock taken, no thread ever blocks.

4.13 Memory Ordering and Memory Barriers

The Memory Ordering Problem

CPUs and compilers reorder instructions for performance. On a single thread this is invisible, but with multiple threads it can cause one thread to see another thread's writes in a different order than they were performed.

Memory order options in C++11 atomics:

- `memory_order_relaxed`: no ordering guarantee. Fastest; use only for counters where order doesn't matter.
- `memory_order_acquire`: all subsequent reads in this thread see all writes by other threads that happened before their matching **release**.
- `memory_order_release`: all previous writes are visible to threads that **acquire** this store.
- `memory_order_seq_cst`: total sequential consistency across all atomic operations across all threads. Slowest; default for `atomic<>`.

Hardware Memory Models

x86 has a **strong** memory model (TSO — Total Store Order): stores from one CPU are seen by others in order. ARM has a **weak** model: almost any reordering is possible. Code that works on x86 without barriers may fail on ARM. Always use C++ atomic semantics instead of relying on hardware model specifics.

4.14 Thread Pools

Thread Pool

Creating a new OS thread for every task is expensive (~1 ms, ~8 MB stack). A **thread pool** pre-creates a fixed set of worker threads that wait for tasks in a queue.

Workflow:

1. Create N threads at startup (where $N \approx$ number of CPU cores).
2. Submit tasks to a shared work queue (protected by mutex + CV).
3. Worker threads dequeue and execute tasks. When done, wait for the next.

Benefits: eliminates thread creation overhead, limits total concurrency, reuses warmed-up thread state (TLS, stack).

Java's `ExecutorService`, Python's `ThreadPoolExecutor`, and most server frameworks use thread pools internally.

4.15 Async/Await and Event Loops

Event-Driven / Async I/O

For I/O-bound workloads (web servers handling thousands of connections), a thread per connection is wasteful (each thread blocks waiting for network data). The event loop model uses **one thread** with **non-blocking I/O**:

1. Register interest in I/O events with the OS (`epoll` on Linux, `kqueue` on macOS, `IOCP` on Windows).
2. The event loop calls `epoll_wait()`: blocks until at least one fd is ready.
3. Dispatch ready events to registered callbacks / resume coroutines.
4. Never block in a callback — that would stall the entire event loop.

async/await (Python, JavaScript, Rust, C#) provides syntactic sugar over coroutines/futures, making async code look sequential. `await` suspends the current coroutine without blocking the thread, yielding control back to the event loop.

```
import asyncio

async def fetch_data():
    await asyncio.sleep(1)   # yields control; event loop runs other tasks
    return 42

async def main():
    results = await asyncio.gather(fetch_data(), fetch_data(), fetch_data())
    # 3 coroutines run concurrently; total time ~1s, not 3s
    print(results)

asyncio.run(main())
```

4.16 The Actor Model

Actor Model

The **actor model** is a concurrency model where:

- The basic unit is an **actor** — a lightweight entity with: a private state, a mailbox (message queue), and a behaviour function.
- Actors communicate **only via messages** — no shared state.
- Each actor processes one message at a time (sequential within an actor, no intra-actor races).
- Actors can create new actors and send messages to others.

No shared mutable state ⇒ no races, no locks needed. Used by: Erlang, Elixir, Akka (JVM), Microsoft Orleans.

Chapter 4 — Key Takeaways

- **Concurrency** is structure; **parallelism** is execution.
- **Race conditions** arise from unguarded access to shared mutable state.
- **Mutexes** provide mutual exclusion; use **spinlocks** only for very short critical sections.
- **Condition variables** let threads wait for conditions without polling.
- Deadlock requires **all four** Coffman conditions; break any one to prevent it. **Lock ordering** is the simplest fix.
- **Atomic CAS** is the foundation of lock-free data structures.
- **Memory ordering** matters on weak-memory architectures (ARM); use C++ atomic semantics.
- **Thread pools** amortise thread creation; **async/event loops** scale better than threads for I/O-bound tasks.

4.17 The Dining Philosophers Problem

Five philosophers sit at a round table with a fork between each pair. To eat, a philosopher needs *both* forks (left and right). They alternate between thinking and eating.

The Problem

Naive solution: each philosopher picks up their left fork, then their right fork. **Deadlock:** if all five simultaneously grab their left fork, none can grab their right fork. They wait forever — classic circular wait.

Solutions

- **Resource ordering**: number forks 1–5. Always grab the lower-numbered fork first. Philosopher 5 grabs fork 1 (not fork 5 first) — breaks circular wait.
- **Allow only $N-1$ philosophers to try eating simultaneously**: use a semaphore initialised to 4. At most 4 try at once; at least one pair of forks is always available.
- **Chandy/Misra (message-passing)**: forks are "dirty" or "clean". A philosopher must request a fork from its neighbour (via message), who sends it only if they're not eating and the fork is dirty. Ensures fairness.
- **Arbitrator (waiter)**: a mutex (the waiter) serialises all fork acquisitions. Simple but reduces concurrency.

4.18 Priority Inversion and the Mars Pathfinder Bug

Priority Inversion

Priority inversion occurs when a high-priority task is blocked waiting for a resource held by a low-priority task — and a medium-priority task preempts the low-priority task, indefinitely delaying the high-priority task.

Sequence:

1. Low-priority task L acquires mutex M.
2. High-priority task H becomes runnable, preempts L. H tries to lock M — blocked.
3. Medium-priority task M (no need for mutex) runs, preempting L.
4. L can't run (preempted by M), so H waits indefinitely for M to release the mutex — even though H has higher priority than M.

Mars Pathfinder (1997)

The Mars Pathfinder lander experienced watchdog resets (system reboots) due to priority inversion between:

- Low-priority: meteorological data gathering task (held a shared bus mutex).
- Medium-priority: communications task (no need for the mutex).
- High-priority: information bus distribution task (needed the mutex).

The high-priority task was starved, missed its deadline, the watchdog tripped, system reset. Fixed remotely by enabling **priority inheritance** on the mutex.

Priority Inheritance Protocol (PIP)

When high-priority task H blocks on a mutex held by low-priority task L, L's priority is **temporarily raised** to H's priority until L releases the mutex. This prevents medium-priority tasks from preempting L.
POSIX: `pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_INHERIT)`.
Priority Ceiling Protocol (PCP): each mutex has a ceiling = the highest priority of any task that may lock it. A task's priority is raised to the ceiling whenever it holds the mutex. Prevents priority inversion and deadlock simultaneously. Used in real-time systems.

4.19 RCU — Read-Copy-Update

RCU (Read-Copy-Update)

RCU is a synchronisation mechanism for **read-heavy** data structures where reads are extremely frequent and must have zero overhead, while writes are rare.
Key insight: in RCU, readers never acquire any lock.

- **Readers**: call `rcu_read_lock()` (disables preemption on that CPU, no lock acquisition) and `rcu_read_unlock()`. During the read-side critical section, the data they're reading cannot be freed.
- **Writers**: create a **new copy** of the modified data structure. Atomically update the pointer to the new version. Then wait for a **grace period** (all current readers finish), then free the old version.

RCU Write Sequence

```
1. Writer: old_ptr = global_ptr           // snapshot old version
2. Writer: new_node = copy(old_ptr)       // copy and modify
3. Writer: rcu_assign_pointer(global_ptr, new_node)  // atomic publish
4. Writer: synchronize_rcu()              // wait for all readers to finish
5. Writer: kfree(old_ptr)                 // safe to free now
```

RCU is pervasive in the Linux kernel: routing tables, network protocol handlers, VFS directory entry cache (dcache), and more. For kernel code, reader-side overhead is essentially zero (just disabling preemption). No cache line bouncing from mutex operations on the read path.

4.20 Lock-Free Queue Implementation

Michael-Scott Queue (Lock-Free SPSC / MPMC)

A lock-free FIFO queue using CAS. The queue has a **head** (dequeue) and **tail** (enqueue) pointer. Each node has a `next` pointer.

```
template<typename T>
class LockFreeQueue {
    struct Node {
        T data;
        std::atomic<Node*> next{nullptr};
    };
    std::atomic<Node*> head, tail;
public:
    LockFreeQueue() {
        Node* dummy = new Node{};  // sentinel node
        head.store(dummy); tail.store(dummy);
    }
    void enqueue(T val) {
        Node* node = new Node{val};
        Node* prev_tail;
        while (true) {
            prev_tail = tail.load();
            Node* next = prev_tail->next.load();
            if (prev_tail == tail.load()) {          // tail hasn't changed
                if (next == nullptr) {               // tail is truly last
                    if (prev_tail->next.compare_exchange_weak(next, node))
                        break;                       // linked node in
                } else {
                    tail.compare_exchange_weak(prev_tail, next); // advance tail
                }
            }
```

```
        }
        tail.compare_exchange_weak(prev_tail, node); // swing tail to new node
    }
    bool dequeue(T& result) {
        while (true) {
            Node* h = head.load();
            Node* t = tail.load();
            Node* next = h->next.load();
            if (h == head.load()) {
                if (h == t) {
                    if (next == nullptr) return false; // empty
                    tail.compare_exchange_weak(t, next);
                } else {
                    result = next->data;
                    if (head.compare_exchange_weak(h, next)) {
                        delete h;  // safe: h was the dummy
                        return true;
                    }
                }
            }
        }
    }
};
```

ABA Problem

CAS can fail due to the **ABA problem**: pointer goes from A to B and back to A; the CAS sees A and succeeds, unaware of the intermediate change. Solutions: tagged pointers (store a version number in unused pointer bits), hazard pointers, or epoch-based reclamation. The above implementation has an ABA issue in the `delete h` step — use hazard pointers in production.

4.21 Go's Goroutines and Channels (CSP Model)

Goroutines

A **goroutine** is Go's lightweight thread: starts with a ∼2 KB stack (grows as needed up to 1 GB), scheduled by Go's runtime M:N scheduler (multiplexing M goroutines onto N OS threads). Creating a goroutine costs ∼2–3 µs vs. ∼1–2 ms for an OS thread.
Go's scheduler uses **work-stealing**: if an OS thread's run queue is empty, it steals goroutines from other threads' queues. Goroutines are descheduled on blocking system calls (the OS thread is handed off to another goroutine; when the syscall completes, the goroutine is re-queued).

Channels (CSP)

Go implements Tony Hoare's **Communicating Sequential Processes (CSP)**: "Do not communicate by sharing memory; instead, share memory by communicating."
Channels are typed, synchronised conduits:

```

// Producer goroutine
go func() {
    for i := 0; i < 100; i++ {
        ch <- i              // send; blocks if buffer full
    }
    close(ch)
}()

// Consumer goroutine
go func() {
    for v := range ch {    // receive until closed
        fmt.Println(v)
    }
}()
```

select statement: multiplex over multiple channels (like `epoll` for channels).

4.22 Java Virtual Threads (Project Loom)

Java Virtual Threads (Java 21+)

Before Project Loom, Java's threading model was 1:1 (one Java thread = one OS thread). Creating a thread per connection was limited to ~10,000 threads before memory and scheduling overhead became prohibitive. **Virtual threads** (Project Loom, Java 21 GA) are lightweight threads scheduled by the JVM on a pool of **carrier threads** (OS threads). When a virtual thread blocks (e.g., on a socket read), the JVM unmounts it from the carrier thread, allowing the carrier to run another virtual thread. When the I/O completes, the virtual thread is remounted on a (possibly different) carrier.

```
try (var executor = Executors.newFixedThreadPool(200)) {
    for (var request : requests)
        executor.submit(() -> handleRequest(request));  // blocks OS thread on I/O
}

// New: Virtual threads (Java 21)
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (var request : requests)
        executor.submit(() -> handleRequest(request));  // unmounts on I/O, scales to millions
}
```

No async/await, no callbacks, no reactive programming. Write blocking-style code that scales like async code. Virtual threads cost ~1 KB of heap vs. ~1 MB for platform threads.

4.23 io_uring: The Modern Linux Async I/O Interface

io_uring (Linux 5.1+)

Traditional async I/O on Linux (`aio_read`, `libaio`) has limitations: only works for O_DIRECT files, has poor buffered I/O support, and requires syscalls per operation.
io_uring uses two **ring buffers** shared between user space and the kernel (via `mmap`) to submit I/O operations and receive completions **without system calls** in the steady state:

- **SQ** (Submission Queue): user writes SQEs (Submission Queue Entries) describing I/O operations (read, write, accept, send, recv, ...).
- **CQ** (Completion Queue): kernel writes CQEs (Completion Queue Entries) when operations complete.
- User submits a batch with one `io_uring_enter()` syscall (or zero syscalls in polling mode where the kernel spins checking the SQ).

Throughput: ~1.7 million 4KB IOPS (vs. ~700K for traditional async I/O) on NVMe SSDs. Supports not just file I/O but also sockets, splice, futex, timers.

```
#include <liburing.h>

struct io_uring ring;
io_uring_queue_init(32, &ring, 0);          // 32-entry ring

// Prepare a read operation
struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);
io_uring_prep_read(sqe, fd, buf, 4096, 0);
sqe->user_data = 42;                        // tag to identify this op

io_uring_submit(&ring);                     // one syscall submits all pending SQEs

// Later: collect completions
struct io_uring_cqe *cqe;
io_uring_wait_cqe(&ring, &cqe);
if (cqe->res > 0) printf("Read %d bytes (op %llu)\n", cqe->res, cqe->user_data);
io_uring_cqe_seen(&ring, cqe);              // advance CQ head

io_uring_queue_exit(&ring);
```

Chapter 4 Additional — Key Takeaways

- **Dining Philosophers**: break circular wait via resource ordering or arbitration.
- **Priority inversion** caused the Mars Pathfinder to reboot. Fix with priority inheritance (`PTHREAD_PRIO_INHERIT`).
- **RCU** gives zero-overhead reads at the cost of slightly complex write paths. The go-to for read-heavy kernel data structures.
- **Lock-free queues** use CAS loops; beware the ABA problem.
- **Go goroutines** + channels implement CSP — cheap concurrency units with structured communication.
- **Java Virtual Threads** (Java 21) allow 1-thread-per-request coding style that scales to millions of concurrent tasks.
- **io_uring** is the state-of-the-art Linux async I/O: ring buffers, near-zero syscall overhead, 2x+ throughput vs. `aio`.

Chapter 5

Latency & Throughput

Performance is not one number — it's a multidimensional profile. Two systems can have the same average response time but radically different tail latencies. This chapter gives you the vocabulary and the laws to reason about performance quantitatively.

5.1 Latency vs. Throughput

Latency

Latency is the time from when a request is issued to when the response is received (or the operation completes). Also called **response time**. Units: nanoseconds, microseconds, milliseconds, seconds.
Examples:

- L1 cache hit latency: ~1 ns
- Database query latency: 1–100 ms
- HTTP API call latency: 10–1000 ms

Throughput

Throughput is the number of operations completed per unit time. Also called **bandwidth** in storage/network contexts. Units: requests/sec (RPS), transactions/sec (TPS), bytes/sec.
Examples:

- A web server handling 50,000 RPS
- A NVMe SSD writing at 3 GB/s
- A CPU executing 3×10^9 instructions/sec

Latency and Throughput Trade-offs

Optimising for one often hurts the other. Example: **batching** increases throughput (process 1000 records at once instead of 1) but increases latency for each individual record (it waits in the batch). **Always know which metric your users care about** before optimising.

5.2 Bandwidth vs. Throughput

These terms are often used interchangeably but have a precise distinction:

Bandwidth vs. Throughput

- **Bandwidth**: the *theoretical maximum* rate of a channel (e.g., a 10 Gbps Ethernet link).
- **Throughput**: the *actual achieved* rate, which is always ≤ bandwidth due to overhead, congestion, and errors.

A 10 Gbps link with TCP overhead, packet retransmissions, and ACK traffic might achieve 7–8 Gbps actual throughput.

5.3 The Latency Numbers Every Engineer Should Know

These numbers, famously popularised by Jeff Dean (Google), give an intuitive feel for the relative cost of different operations:

Approximate Latency Numbers (2024)

Operation	Latency
L1 cache reference	1 ns
Branch misprediction	5 ns
L2 cache reference	4 ns
Mutex lock/unlock	25 ns
Main memory reference (DRAM)	100 ns
Compress 1 KB with Snappy	3 µs
Send 1 KB over 1 Gbps network	10 µs
Read 4 KB randomly from NVMe SSD	100 µs
Read 1 MB sequentially from memory	250 µs
Round trip within same datacenter	500 µs
Read 1 MB sequentially from NVMe SSD	1 ms
Disk seek (HDD)	10 ms
Read 1 MB sequentially from HDD	20 ms
Send packet CA → Netherlands → CA	150 ms

Key Ratios to Internalize

- Memory is **100× slower** than L1 cache.
- NVMe SSD is **1,000× slower** than memory for random reads.
- HDD random seek is **100,000× slower** than memory.
- Cross-datacenter network is **1,000,000× slower** than L1 cache.

These ratios explain why caching at every level is so important.

5.4 Little's Law

Little's Law

For any stable system (where arrival rate = departure rate):

$$L = \lambda \cdot W$$

- L = average number of items in the system (queue + service)
- λ = average arrival rate (items/sec)
- W = average time an item spends in the system (latency)

This law is **remarkably general** — it requires no assumptions about arrival distribution, service time distribution, or number of servers.

Little's Law in Practice

Scenario: A web server handles 1,000 RPS and each request takes an average of 200 ms to complete.
$L = \lambda \cdot W = 1000$ req/s $\times$ 0.2 s = **200 concurrent requests** in the system at any moment.
This tells us we need capacity for 200 concurrent threads/connections — not just "handle 1000 RPS."
Conversely: if we observe 200 concurrent requests and 1000 RPS, we know average latency is $W = L/\lambda = 200/1000 = 0.2$s.

5.5 Amdahl's Law

Amdahl's Law

Adding more processors doesn't always give proportional speedup. If a fraction f of the program is inherently sequential (cannot be parallelised), the maximum speedup S achievable with N processors is:

$$S(N) = \frac{1}{f + \frac{1-f}{N}}$$

As $N \to \infty$: $S_{\max} = \frac{1}{f}$

If 10% of your program is sequential, the maximum possible speedup is **10**×, no matter how many cores you add.

Amdahl in Practice

Sequential fraction f	N=2	N=4	N=8	N=∞
5%	1.90×	3.48×	5.93×	20×
10%	1.82×	3.08×	4.71×	10×
20%	1.67×	2.50×	3.33×	5×
50%	1.33×	1.60×	1.78×	2×

Amdahl's Pessimism

Amdahl's Law assumes fixed problem size. In practice, as you add hardware, you often also increase the problem size (more users, more data). **Gustafson's Law** (next section) addresses this.

5.6 Gustafson's Law

Gustafson's Law

If you scale the problem size with the number of processors (the parallel portion grows to keep all processors busy):

$$S(N) = N - f \cdot (N - 1)$$

where f is the *serial fraction* and N is the number of processors.
Key insight: For large problems, parallelism scales nearly linearly — even if a small serial fraction exists — because the parallel work dominates.

Gustafson's Law better models real-world scenarios: when you buy 100 servers, you also take on 100× more data. The serial bottleneck (e.g., coordinator logic) may remain fixed while the parallel work scales. Speedup is then nearly linear in N.

5.7 Tail Latency and Percentiles

Averages are misleading. A system where 99 out of 100 requests complete in 1 ms and 1 in 100 takes 10 seconds has a great average (~1.1 ms) but catastrophic user experience for that 1%.

Percentiles

- **p50 (median)**: 50% of requests complete within this time.
- **p99**: 99% of requests complete within this time. The slowest 1%.
- **p99.9** (the "nines"): 99.9% complete within this time. 1 in 1,000 requests is slower.
- **p99.99**: 1 in 10,000 is slower.

Production systems typically define SLOs (Service Level Objectives) using high percentiles: "p99 latency < 100 ms."

The Tail Latency Amplification Problem

In a microservices system, a single request may fan out to 100 downstream services. Even if each service has p99 = 1 ms, the probability that *at least one* of 100 is slow is: $1 - (0.99)^{100} \approx 63\%$
So 63% of your composite requests will be slow! **Hedged requests** (send the same request to two backends, use whichever responds first) and **timeout/retry strategies** are essential at scale.

5.8 Queueing Theory Basics

The M/M/1 Queue

A classic model: requests arrive according to a Poisson process at rate λ, are served exponentially with rate μ (one server). Define **utilisation** $\rho = \lambda/\mu$.
Key results:

- Average queue length: $L = \dfrac{\rho}{1-\rho}$
- Average time in system: $W = \dfrac{1}{\mu(1-\rho)}$
- **The queue blows up as** $\rho \to 1$: a server at 90% utilisation has 9× the average latency of a server at 50% utilisation.

The Utilisation-Latency Cliff

The M/M/1 model predicts that latency rises **non-linearly** with utilisation. At 50% utilisation, average latency is ≈ 2× the service time. At 90%, it's ≈ 10×. At 99%, it's ≈ 100×.
In practice: keep your servers at <70% average utilisation to maintain low tail latency. Use **auto-scaling** to shed load before the cliff.

5.9 Profiling and Benchmarking

Profiling

Profiling identifies where your program actually spends its time. Never optimise without profiling first — you will almost certainly guess wrong.
Tools:

- **gprof**: compile-time instrumentation, reports per-function time.
- **perf** (Linux): sampling profiler, no recompile needed. `perf record ./prog; perf report` shows hot functions.
- **Valgrind / Cachegrind**: cache miss analysis.
- **Flame graphs**: visualise where time is spent hierarchically.
- **eBPF / BPF tools**: production profiling with near-zero overhead.

Benchmarking Best Practices

- **Warm up**: run a few iterations before measuring (allow JIT, cache warm-up).
- **Isolate**: benchmark one thing at a time; control all other variables.
- **Measure the right thing**: wall-clock time, not CPU time, if I/O is involved. Measure p99, not just mean.
- **Beware dead code elimination**: compilers can optimise away benchmarked code if results are unused. Use `DoNotOptimize()` / volatile tricks.
- **Run multiple times**: take the median of many runs, not just one.

5.10 Performance Optimisation Techniques

General Optimisation Hierarchy

1. **Algorithm and data structure** — the biggest gains. $O(n \log n)$ beats $O(n^2)$ regardless of constants. Use the right data structure.
2. **Reduce I/O**: batch writes, use async I/O, add caching.
3. **Reduce allocations**: reuse objects, use pools, prefer stack allocation.
4. **Improve cache behaviour**: sequential access, smaller data structures, structure-of-arrays vs. array-of-structures.
5. **Reduce synchronisation**: lock-free data structures, partitioned data to avoid sharing.
6. **Parallelise**: split work across cores / machines.
7. **SIMD and low-level tuning**: usually last resort; hand-vectorise hot inner loops.

Cache-Friendly Data Layout: SoA vs. AoS

```
// Array of Structures (AoS) -- bad for iterating one field
struct Particle { float x, y, z, mass; };
Particle particles[N];
// Iterating only x: loads mass, y, z into cache too (wasted)

// Structure of Arrays (SoA) -- cache-friendly when iterating one field
struct Particles { float x[N], y[N], z[N], mass[N]; };
// Iterating only x: perfect spatial locality, no wasted cache lines
// Common in game engines, HPC, physics simulations
```

Chapter 5 — Key Takeaways

- **Latency** is time for one operation; **throughput** is operations per second. They trade off.
- Memorise the **latency numbers**: memory is 100× L1, disk seek is 10^5× L1.
- **Little's Law**: concurrency = rate × latency. Knowing two gives you the third.
- **Amdahl's Law**: serial fractions create hard speedup ceilings. Parallelising 90% gives at most 10× speedup.
- Use **percentiles** (p99, p99.9), not averages, for latency SLOs.
- Queueing theory: **latency explodes near 100% utilisation**. Keep servers at <70% average load.
- **Profile before optimising**. Optimise algorithms before micro-tuning.

5.11 CPU Performance Counters and the `perf` Tool

Hardware Performance Counters

Modern CPUs have **Performance Monitoring Units (PMU)** with registers that count hardware events: instructions retired, cycles, cache hits/misses, branch mispredictions, TLB misses, etc. The OS exposes these via `perf_event_open()`.
Key metrics:

- **IPC** (Instructions Per Cycle): >3 is excellent; <1 suggests memory or branch bottlenecks.
- **LLC miss rate**: last-level cache miss rate. High miss rate = memory bound.
- **Branch misprediction rate**: >1% is significant for tight loops.
- **CPI** (Cycles Per Instruction) = 1/IPC.

Using perf

```
# Profile a program (statistical sampling)
perf record -g ./my_program          # record with call graph
perf report                          # interactive TUI browser

# Count hardware events
perf stat -e cycles,instructions,cache-misses,branch-misses ./my_program
# Output:
#  5,234,100,000    cycles
#  8,109,400,000    instructions      #   1.55  insn per cycle
#    123,400,000    cache-misses      #   2.45% of all cache refs
#      1,200,000    branch-misses     #   0.12% of all branches

# Profile system-wide for 5 seconds
perf top -a -g --sleep 5

# Find cache misses by source line
perf record -e cache-misses -g ./prog && perf report --sort=dso,sym
```

Flame Graphs

Brendan Gregg's **flame graphs** visualise profiling data. The x-axis is **alphabetically sorted** (not time); x-width represents time spent in that function + its callees. The y-axis is the call stack depth.

```
# Generate a flame graph from perf data
perf record -F 99 -g ./my_program
perf script | stackcollapse-perf.pl | flamegraph.pl > flame.svg
# Open flame.svg in a browser: click to zoom into hot functions
```

Flame graphs reveal: top-heavy functions (wide frames at the top = where CPU spends time), unexpected call chains, and GC pauses (wide "GC" frames).

5.12 Mechanical Sympathy

Mechanical Sympathy

The term, coined by Martin Thompson (LMAX Disruptor), means: *understanding the machine's mechanical properties to write software that works with them, not against them.*
Key principles:

- **Cache line awareness**: group related data together; avoid false sharing.
- **Branch-free code**: replace branches with arithmetic in inner loops.
- **Memory access patterns**: sequential over random; prefetch when predictable.
- **Minimise allocations**: avoid GC pauses; reuse objects; use pools.
- **Thread affinity**: pin threads to cores; avoid NUMA crossings.
- **Lock-free algorithms**: avoid mutex overhead in hot paths.
- **Batching**: amortise syscall and I/O overhead.

LMAX Disruptor

The LMAX Disruptor is a lock-free inter-thread messaging library achieving >25 million events/sec on a single machine (compared to ~1 million with a traditional `ArrayBlockingQueue`).
Key techniques:

- **Ring buffer**: pre-allocated, fixed-size, power-of-two. Cache-friendly, no GC pressure (reuses slots).
- **Sequence numbers**: producers and consumers track positions with `AtomicLong` sequence numbers instead of head/tail pointers.
- **Padding**: each sequence number is padded to 128 bytes (2 cache lines) to prevent false sharing between producer and consumer sequences.
- **Wait strategies**: configurable from "busy spin" (lowest latency, 100% CPU) to "blocking wait" (lowest CPU, highest latency).

5.13 Zero-Copy I/O: sendfile and splice

The Problem with Traditional File Serving

Serving a file over a socket with the traditional approach:

1. `read(file_fd, buf, n)`: kernel copies file data from page cache to kernel buffer, then to **user-space buffer**. (Copy 1)
2. `write(socket_fd, buf, n)`: kernel copies from user-space buffer to socket send buffer. (Copy 2)

Two data copies + two context switches. For a 100 MB file transfer: wasteful.

sendfile() — Zero-Copy

`sendfile(out_fd, in_fd, offset, count)`: the kernel copies data directly from the file's page cache to the socket buffer — **no user-space buffer, no user-space copy**. Data never leaves the kernel.
With **scatter-gather DMA**: the NIC reads directly from the page cache (DMA'd to the NIC), zero copies of the payload at all. Only the TCP/IP header is generated by the CPU.

```
#include <sys/sendfile.h>
int file_fd = open("bigfile.bin", O_RDONLY);
off_t offset = 0;
sendfile(socket_fd, file_fd, &offset, file_size);  // zero-copy!
```

Used by: nginx (`sendfile on;` in config), Apache, most web servers. Throughput improvement: 2–4× for large file transfers.

splice() — Pipe-Based Zero-Copy

`splice()` transfers data between two file descriptors via a pipe, without copying to user space. Can move data between sockets, files, and pipes entirely in the kernel.
`tee()` duplicates pipe data (like the `tee` shell command) without consuming it — useful for logging raw network traffic while also forwarding it.

5.14 Kernel Bypass Networking: DPDK and RDMA

DPDK (Data Plane Development Kit)

The Linux network stack adds ∼3–5 µs of overhead per packet (kernel processing, context switches, socket buffer copies). For high-frequency trading, 5G packet cores, and network functions, this is too slow.
DPDK bypasses the kernel entirely:

- NIC is taken from the kernel driver and managed by a **user-space driver**.
- Packets are read directly from the NIC's DMA ring buffer in user space via **polling** (no interrupts — a dedicated core spins 100% of the time).
- Packet processing pipelines run in user space; no syscalls, no copies.

Throughput: **up to 200 Gbps** on a single CPU core. Latency: <1 µs. Cost: dedicated CPU cores that can't do other work.
Used by: Cisco, Juniper, telco packet cores, Cloudflare, NYSE/CME trading systems.

RDMA (Remote Direct Memory Access)

RDMA allows a machine to **read or write another machine's memory directly** — without involving the remote CPU at all. The remote CPU is not interrupted; no context switch, no data copy, no socket buffer.

- Latency: ∼1 µs round-trip (vs. ∼50 µs for TCP loopback).
- Bandwidth: 100–400 Gbps.
- Protocols: InfiniBand, RoCE (RDMA over Converged Ethernet), iWARP.

Used in: HPC clusters (MPI over InfiniBand), distributed storage (Ceph, Lustre), cloud high-performance networking (Azure, AWS placement groups with EFA), and in-memory databases (RAMCloud, FaRM).

Chapter 5 Additional — Key Takeaways

- **perf stat** gives you IPC, cache miss rates, branch mispredictions in seconds. Always measure before optimising.
- **Flame graphs** make profiler output human-readable. Learn to read them.
- **Mechanical sympathy**: write code that works with CPU caches, branch predictors, and memory architecture, not against them.
- **Zero-copy** (`sendfile`): eliminates user-space buffer copies for file serving, giving 2–4x throughput improvement.
- **DPDK** bypasses the kernel for < 1 µs packet latency at the cost of dedicated polling cores.
- **RDMA** enables memory access across machines without involving the remote CPU: ∼1 µs vs. ∼50 µs for TCP.

Chapter 6

Design Patterns

Design patterns are reusable solutions to recurring software design problems. They're not copy-paste code — they're a shared vocabulary and a catalogue of proven approaches. Knowing them lets you recognise familiar problems and communicate solutions concisely with other engineers.

The classic reference is the Gang of Four book (GoF): "Design Patterns: Elements of Reusable Object-Oriented Software" (Gamma, Helm, Johnson, Vlissides, 1994). Patterns are grouped into three categories: **Creational**, **Structural**, and **Behavioural**.

6.1 Why Design Patterns?

The Problem Patterns Solve

As software grows, recurring design problems appear:

- How do I ensure only one instance of this class exists?
- How do I add behaviour to objects without modifying their class?
- How do I decouple notification senders from receivers?
- How do I create objects without knowing their concrete type?

Patterns capture the collective experience of expert software designers. They provide a solution *template* — not a finished implementation — that must be adapted to the specific context.

Anti-Pattern: Pattern Overuse

Patterns are tools, not goals. Forcing a pattern where it doesn't fit produces unnecessarily complex code. The best engineers know *when not* to use a pattern. Simple, direct code is almost always preferable to a clever pattern if the pattern's benefits don't apply.

6.2 Singleton Pattern

Singleton

Intent: Ensure a class has at most one instance, and provide a global access point to it.
Use when: Exactly one object must coordinate actions across the system (e.g., logger, configuration manager, thread pool, connection pool).

```
// Thread-safe Singleton using C++11 static local (Meyers' Singleton)
class Logger {
public:
    static Logger& instance() {
        static Logger inst;    // created once, on first call, thread-safe in C++11
        return inst;
    }
    void log(const std::string& msg) { /* ... */ }
private:
    Logger() {}                        // private constructor
    Logger(const Logger&) = delete;    // no copy
```

```
    Logger& operator=(const Logger&) = delete;
};

// Usage:
Logger::instance().log("Hello");
```

Singleton Pitfalls

- Makes unit testing hard (can't inject mock, global state persists between tests).
- Hidden dependency: callers implicitly depend on a global.
- Concurrency: naive implementations have race conditions on first creation.
- Consider **dependency injection** (pass the object explicitly) as an alternative.

6.3 Factory Method Pattern

Factory Method

Intent: Define an interface for creating an object, but let subclasses decide which class to instantiate. Decouple object creation from use.
Use when: A class cannot anticipate the type of objects it must create, or you want subclasses to control object creation.

```
class Shape { public: virtual void draw() = 0; virtual ~Shape() {} };
class Circle : public Shape { public: void draw() override { /* ... */ } };
class Square : public Shape { public: void draw() override { /* ... */ } };

class ShapeFactory {
public:
    static std::unique_ptr<Shape> create(const std::string& type) {
        if (type == "circle") return std::make_unique<Circle>();
        if (type == "square") return std::make_unique<Square>();
        throw std::invalid_argument("Unknown shape: " + type);
    }
};

// Usage: caller doesn't need to know Circle or Square exist
auto shape = ShapeFactory::create("circle");
shape->draw();
```

6.4 Abstract Factory Pattern

Abstract Factory

Intent: Provide an interface for creating *families* of related or dependent objects without specifying their concrete classes.
Use when: A system must be independent of how its products are created, and you need to ensure that a family of products is used together (e.g., a GUI toolkit that creates consistent Windows-native or macOS-native widgets).

```
// Product interfaces
class Button { public: virtual void click() = 0; };
class TextBox { public: virtual void input(const std::string&) = 0; };

// Abstract factory
class UIFactory {
public:
    virtual std::unique_ptr<Button>  createButton()  = 0;
    virtual std::unique_ptr<TextBox> createTextBox() = 0;
};

// Concrete factory for each platform
class WindowsFactory : public UIFactory {
    std::unique_ptr<Button>  createButton()  override { return std::make_unique<WinButton>(); }
    std::unique_ptr<TextBox> createTextBox() override { return std::make_unique<WinTextBox>(); }
};
class MacFactory : public UIFactory { /* ... */ };

// Client code: works with any factory
void buildUI(UIFactory& factory) {
    auto btn = factory.createButton();
```

```
    auto txt = factory.createTextBox();
    btn->click();
}
```

6.5 Builder Pattern

Builder

Intent: Separate the construction of a complex object from its representation, allowing the same construction process to create different representations.
Use when: An object requires many optional parameters (avoid telescoping constructors), or construction involves multiple steps.

```
class HttpRequest {
public:
    std::string url, method, body;
    std::map<std::string, std::string> headers;
    int timeout_ms = 5000;

    class Builder {
        HttpRequest req;
    public:
        Builder& url(const std::string& u)    { req.url = u;       return *this; }
        Builder& method(const std::string& m) { req.method = m;    return *this; }
        Builder& header(const std::string& k, const std::string& v) {
            req.headers[k] = v; return *this;
        }
        Builder& body(const std::string& b)   { req.body = b;      return *this; }
        Builder& timeout(int ms)              { req.timeout_ms = ms; return *this; }
        HttpRequest build() { return req; }
    };
};

// Usage: readable, all parameters explicit, order doesn't matter
auto req = HttpRequest::Builder()
    .url("https://api.example.com/data")
    .method("POST")
    .header("Content-Type", "application/json")
    .body("{\"key\":\"value\"}")
    .timeout(3000)
    .build();
```

6.6 Prototype Pattern

Prototype

Intent: Create new objects by cloning an existing object (the prototype), rather than creating from scratch.
Use when: Object creation is expensive (e.g., deep DB query, complex initialisation) and you frequently need near-identical objects, or when the class to instantiate is specified at runtime.

```
class Config {
public:
    std::map<std::string, std::string> settings;  // expensive to load from file
    virtual Config* clone() const { return new Config(*this); }  // prototype
};

// Instead of loading config from disk each time:
Config* base = loadConfigFromDisk();
Config* request1_config = base->clone();   // fast copy
Config* request2_config = base->clone();   // fast copy
request2_config->settings["timeout"] = "3000";  // per-request override
```

6.7 Adapter Pattern

Adapter (Wrapper)

Intent: Convert the interface of a class into another interface that clients expect. Allows incompatible interfaces to work together.
Real-world analogy: A power adapter lets a US plug fit a European socket. The plug's interface is incompatible; the adapter translates without changing either side.
Use when: You want to use an existing class but its interface doesn't match what you need, and you can't (or don't want to) modify it.

```
// Third-party JSON library you can't modify
class LegacyJsonParser {
public:
    std::string parse_legacy(const char* data, int len) { /* ... */ }
};

// Your codebase expects this interface
class JsonParser { public: virtual std::string parse(const std::string& s) = 0; };

// Adapter wraps the legacy class and exposes your interface
class JsonParserAdapter : public JsonParser {
    LegacyJsonParser legacy;
public:
    std::string parse(const std::string& s) override {
        return legacy.parse_legacy(s.c_str(), s.size());
    }
};
```

6.8 Decorator Pattern

Decorator

Intent: Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.
Use when: You need to add behaviour to individual objects, not an entire class, and adding subclasses for every combination would be an explosion of classes.

```
class Coffee { public: virtual double cost() = 0; virtual std::string desc() = 0; };

class SimpleCoffee : public Coffee {
    double cost() override { return 1.0; }
    std::string desc() override { return "Coffee"; }
};

// Decorator base
class CoffeeDecorator : public Coffee {
protected: Coffee* base;
public: CoffeeDecorator(Coffee* c) : base(c) {}
};

class MilkDecorator : public CoffeeDecorator {
public:
    MilkDecorator(Coffee* c) : CoffeeDecorator(c) {}
    double cost() override { return base->cost() + 0.25; }
    std::string desc() override { return base->desc() + ", Milk"; }
};

class SugarDecorator : public CoffeeDecorator {
public:
    SugarDecorator(Coffee* c) : CoffeeDecorator(c) {}
    double cost() override { return base->cost() + 0.10; }
    std::string desc() override { return base->desc() + ", Sugar"; }
};

// Compose at runtime
Coffee* c = new SugarDecorator(new MilkDecorator(new SimpleCoffee()));
// c->cost() = 1.35, c->desc() = "Coffee, Milk, Sugar"
```

6.9 Facade Pattern

Facade

Intent: Provide a simplified, unified interface to a complex subsystem. The facade hides the complexity and interdependencies of the subsystem's components.
Use when: A subsystem is complex and clients only need a simple interface, or you want to layer a subsystem to reduce dependencies.

```
// Complex subsystem (many classes, intricate interactions)
class VideoDecoder { void decode(const File&); };
class AudioDecoder { void decode(const File&); };
class CodecRegistry { Codec* find(const std::string& fmt); };
class VideoPlayer   { void play(Frame*, Audio*); };

// Facade: simple interface hiding the complexity
class MediaPlayer {
    VideoDecoder vd; AudioDecoder ad; CodecRegistry cr; VideoPlayer vp;
public:
    void play(const std::string& filename) {
        // orchestrates all subsystem components internally
        auto codec = cr.find(getFormat(filename));
        vd.decode(File(filename));
        ad.decode(File(filename));
        vp.play(/* frames */, /* audio */);
    }
};
// Client just calls: MediaPlayer().play("movie.mp4")
```

6.10 Proxy Pattern

Proxy

Intent: Provide a surrogate or placeholder for another object to control access to it.
Types of proxy:

- **Remote proxy**: represents an object in a different address space (e.g., gRPC stubs, Java RMI).
- **Virtual proxy**: defers expensive creation until first use (lazy initialisation).
- **Protection proxy**: controls access based on permissions.
- **Caching proxy**: caches results of expensive operations.

```
class Image { public: virtual void display() = 0; };

class RealImage : public Image {
    std::string filename;
public:
    RealImage(const std::string& f) : filename(f) {
        loadFromDisk();  // expensive!
    }
    void display() override { /* render */ }
    void loadFromDisk() { /* slow disk read */ }
};

// Virtual proxy: delays loading until actually needed
class ImageProxy : public Image {
    std::string filename;
    RealImage* real = nullptr;
public:
    ImageProxy(const std::string& f) : filename(f) {}  // no load yet
    void display() override {
        if (!real) real = new RealImage(filename);  // load on first use
        real->display();
    }
};
```

6.11 Observer Pattern

Observer (Publish-Subscribe)

Intent: Define a one-to-many dependency between objects: when one object (the **Subject/Publisher**) changes state, all its dependents (**Observers/Subscribers**) are notified automatically.
Use when: Changes to one object require updating others, and you don't know in advance how many or who they are. Decouples the publisher from subscribers.

```cpp
class Observer { public: virtual void update(const std::string& event) = 0; };

class EventBus {
    std::map<std::string, std::vector<Observer*>> subscribers;
public:
    void subscribe(const std::string& event, Observer* obs) {
        subscribers[event].push_back(obs);
    }
    void publish(const std::string& event) {
        for (auto* obs : subscribers[event]) obs->update(event);
    }
};

class Logger : public Observer {
    void update(const std::string& e) override { std::cout << "LOG: " << e << "\n"; }
};
class Metrics : public Observer {
    void update(const std::string& e) override { /* increment counter */ }
};

// Usage
EventBus bus;
Logger logger; Metrics metrics;
bus.subscribe("user.login", &logger);
bus.subscribe("user.login", &metrics);
bus.publish("user.login");  // both Logger and Metrics are notified
```

6.12 Strategy Pattern

Strategy

Intent: Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
Use when: Multiple variants of an algorithm exist, and you want to select the variant at runtime without a cascade of `if/else` or `switch` statements.

```cpp
class SortStrategy { public: virtual void sort(std::vector<int>& v) = 0; };
class QuickSort   : public SortStrategy { void sort(std::vector<int>& v) override { /* ... */ } };
class MergeSort   : public SortStrategy { void sort(std::vector<int>& v) override { /* ... */ } };
class CountingSort: public SortStrategy { void sort(std::vector<int>& v) override { /* ... */ } };

class Sorter {
    SortStrategy* strategy;
public:
    Sorter(SortStrategy* s) : strategy(s) {}
    void setStrategy(SortStrategy* s) { strategy = s; }
    void sort(std::vector<int>& data) { strategy->sort(data); }
};

// Select strategy based on data characteristics at runtime
Sorter sorter(data.size() > 1000000 ? (SortStrategy*)new MergeSort()
                                    : (SortStrategy*)new QuickSort());
sorter.sort(data);
```

6.13 Command Pattern

Command

Intent: Encapsulate a request as an object, allowing parameterisation of clients with different requests, queueing, logging, and support for undo/redo.
Use when: You need to parameterise objects with actions, support undo, implement transaction logs, or build a task queue.

```cpp
class Command { public: virtual void execute() = 0; virtual void undo() = 0; };

class Document {
    std::string text;
public:
    void insert(int pos, const std::string& s) { text.insert(pos, s); }
    void remove(int pos, int len)              { text.erase(pos, len); }
};

class InsertCommand : public Command {
    Document& doc; int pos; std::string text;
public:
```

```
    InsertCommand(Document& d, int p, const std::string& t) : doc(d), pos(p), text(t) {}
    void execute() override { doc.insert(pos, text); }
    void undo()    override { doc.remove(pos, text.size()); }
};

// Undo stack
std::stack<Command*> history;
Command* cmd = new InsertCommand(doc, 5, "hello");
cmd->execute(); history.push(cmd);

// Undo last command
if (!history.empty()) { history.top()->undo(); history.pop(); }
```

6.14 Template Method Pattern

Template Method

Intent: Define the skeleton of an algorithm in a base class, deferring some steps to subclasses. Subclasses can override specific steps without changing the overall algorithm's structure.
Use when: Multiple classes share the same algorithm structure but differ in some steps.

```
class DataProcessor {
public:
    // Template method -- defines the algorithm structure
    void process() {
        readData();          // step 1
        parseData();         // step 2 -- subclass implements
        analyzeData();       // step 3
        writeResults();      // step 4 -- subclass implements
    }
protected:
    virtual void parseData()   = 0;   // must override
    virtual void writeResults()= 0;   // must override
    void readData()    { /* read from source, same for all */ }
    void analyzeData() { /* common analysis logic */ }
};

class CsvProcessor : public DataProcessor {
    void parseData()    override { /* parse CSV */ }
    void writeResults() override { /* write CSV */ }
};
class JsonProcessor : public DataProcessor {
    void parseData()    override { /* parse JSON */ }
    void writeResults() override { /* write JSON */ }
};
```

6.15 State Pattern

State

Intent: Allow an object to alter its behaviour when its internal state changes. The object will appear to change its class.
Use when: An object's behaviour depends on its state, and must change at runtime based on that state. Avoids large switch/if-else chains on state.

```
class TrafficLight;
class State { public: virtual void handle(TrafficLight& tl) = 0; };

class TrafficLight {
    State* state;
public:
    TrafficLight(State* s) : state(s) {}
    void setState(State* s) { delete state; state = s; }
    void change() { state->handle(*this); }
};

class GreenState : public State {
    void handle(TrafficLight& tl) override {
        std::cout << "GREEN -> YELLOW\n";
        tl.setState(new YellowState());
    }
};
class YellowState : public State {
    void handle(TrafficLight& tl) override {
        std::cout << "YELLOW -> RED\n";
        tl.setState(new RedState());
```

```
    }
};
class RedState : public State {
    void handle(TrafficLight& tl) override {
        std::cout << "RED -> GREEN\n";
        tl.setState(new GreenState());
    }
};
```

6.16 Iterator Pattern

Iterator

Intent: Provide a way to access the elements of a collection sequentially without exposing its underlying representation.
Modern C++ (and Java, Python) bakes this into the language: range-based for loops use iterators behind the scenes. All STL containers provide iterators.

```
// Custom iterator for a binary tree (in-order traversal)
class TreeIterator {
    std::stack<TreeNode*> stk;
    void pushLeft(TreeNode* n) { while(n) { stk.push(n); n = n->left; } }
public:
    TreeIterator(TreeNode* root) { pushLeft(root); }
    bool hasNext() { return !stk.empty(); }
    int next() {
        TreeNode* n = stk.top(); stk.pop();
        pushLeft(n->right);
        return n->val;
    }
};

// Client code doesn't know the tree structure -- just iterates
TreeIterator it(root);
while (it.hasNext()) std::cout << it.next() << " ";
```

6.17 Thread Pool Pattern (Concurrency)

Thread Pool (Concurrency Pattern)

A **thread pool** is a managed collection of pre-created worker threads that execute submitted tasks from a queue. Eliminates per-task thread creation overhead and bounds total concurrency.
Components:

- **Worker threads**: N threads spinning on the task queue.
- **Task queue**: thread-safe queue of pending work items.
- **Submit interface**: clients enqueue tasks; receive a Future/promise for the result.

```
#include <thread>
#include <functional>
#include <queue>
#include <mutex>
#include <condition_variable>

class ThreadPool {
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex mu; std::condition_variable cv;
    bool stop = false;
public:
    ThreadPool(size_t n) {
        for (size_t i = 0; i < n; i++)
            workers.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    { std::unique_lock<std::mutex> lk(mu);
                      cv.wait(lk, [this]{ return stop || !tasks.empty(); });
                      if (stop && tasks.empty()) return;
                      task = std::move(tasks.front()); tasks.pop();
                    }
                    task();
                }
            });
    }
    void submit(std::function<void()> task) {
```

```
        { std::lock_guard<std::mutex> lk(mu); tasks.push(std::move(task)); }
        cv.notify_one();
    }
    ~ThreadPool() {
        { std::lock_guard<std::mutex> lk(mu); stop = true; }
        cv.notify_all();
        for (auto& t : workers) t.join();
    }
};
```

Chapter 6 — Key Takeaways

- **Creational**: Singleton (one instance), Factory (decouple creation), Builder (many optional params), Prototype (clone expensive objects).
- **Structural**: Adapter (bridge incompatible interfaces), Decorator (add behaviour without inheritance), Facade (simplify subsystems), Proxy (control access / lazy init).
- **Behavioural**: Observer (event notification), Strategy (swap algorithms), Command (encapsulate actions / undo), Template Method (skeleton algorithm), State (behaviour changes with state), Iterator (sequential access).
- Patterns are **vocabulary and templates**, not rules. Use only when they solve a real problem in your context.

6.18 SOLID Principles

SOLID is an acronym for five design principles that make object-oriented code more maintainable, extensible, and understandable.

S — Single Responsibility Principle (SRP)

A class should have only one reason to change.
Each class should encapsulate one concept or responsibility. If a class handles both business logic *and* database persistence, a schema change forces a change to the business logic class — they're coupled.

```
// BAD: UserManager handles too much
class UserManager {
    void createUser(User u) { /* business logic */ }
    void saveToDatabase(User u) { /* SQL queries */ }
    void sendWelcomeEmail(User u) { /* SMTP logic */ }
};

// GOOD: separate responsibilities
class UserService  { void createUser(User u); };
class UserRepo     { void save(User u); };
class EmailService { void sendWelcome(User u); };
```

O — Open/Closed Principle (OCP)

Software entities should be open for extension, closed for modification.
Add new functionality by adding new code (new classes, implementations), not by modifying existing, tested code. The classic way: define interfaces, add new implementations.

```
// BAD: must modify existing code to add new shape
double area(Shape s) {
    if (s.type == CIRCLE) return 3.14 * s.r * s.r;
    if (s.type == SQUARE) return s.side * s.side;
    // Must add if-else for every new shape!
}

// GOOD: add new shape by adding a new class, no modification
class Shape     { virtual double area() = 0; };
class Circle    : public Shape { double area() override { return 3.14*r*r; } };
class Square    : public Shape { double area() override { return side*side; } };
class Triangle  : public Shape { double area() override { /* ... */ } };  // new -- no existing code
    changed
```

L — Liskov Substitution Principle (LSP)

Objects of a subclass must be substitutable for objects of the superclass without altering correctness.
A subclass should honour all contracts (preconditions, postconditions, invariants) of its superclass.

```
// LSP VIOLATION:
class Rectangle { virtual void setWidth(int w); virtual void setHeight(int h); };
class Square : public Rectangle {
    void setWidth(int w)  override { width = height = w; }  // violates!
    void setHeight(int h) override { width = height = h; }  // violates!
};
// Code expecting Rectangle behaviour: r.setWidth(5); r.setHeight(3); assert(area()==15)
// Fails for Square! (area would be 9, not 15)
```

Fix: don't make `Square` a subclass of `Rectangle`. They're not substitutable.

I — Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they don't use.
Split large interfaces into smaller, specific ones. A class implementing the interface only needs to implement what it actually uses.

```
// BAD: fat interface
class Worker { virtual void work() = 0; virtual void eat() = 0; virtual void sleep() = 0; };
// RobotWorker is forced to implement eat() and sleep() -- doesn't apply to robots!

// GOOD: segregated interfaces
class Workable { virtual void work() = 0; };
class Feedable { virtual void eat() = 0; };
class Sleepable{ virtual void sleep() = 0; };
class Human    : public Workable, Feedable, Sleepable { /* ... */ };
class Robot    : public Workable { /* only implements work() */ };
```

D — Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details.

```
// BAD: high-level UserService depends on low-level MySQLDatabase
class UserService {
    MySQLDatabase db;  // tightly coupled to MySQL!
    void save(User u) { db.insert(u); }
};

// GOOD: both depend on the abstraction (IDatabase)
class IDatabase { virtual void insert(User u) = 0; };
class MySQLDatabase : public IDatabase { /* ... */ };
class PostgresDatabase : public IDatabase { /* ... */ };

class UserService {
    IDatabase& db;  // depends on abstraction
    UserService(IDatabase& db) : db(db) {}  // injection
    void save(User u) { db.insert(u); }
};
// Switch databases without changing UserService!
```

6.19 Dependency Injection (DI)

Dependency Injection

DI is the practice of **providing** a class's dependencies from the outside rather than having the class construct them internally.
Types:

- **Constructor injection** (preferred): dependencies passed via constructor. Dependencies are explicit, class is always in a valid state.
- **Method/setter injection**: dependencies set via methods after construction. Allows optional dependencies but class can be in invalid state before injection.
- **Field injection** (via reflection, e.g., Spring's `@Autowired`): convenient but hidden dependencies, hard to test.

DI containers (Spring, Guice, Dagger): automatically wire dependencies based on types and annotations. Reduces boilerplate for large applications.

DI Makes Testing Trivial

```
// Without DI: hard to test without hitting real database
class UserService { UserService() { db = new PostgresDB("prod.host"); } };

// With DI: inject a mock for testing
class UserService { UserService(IDatabase& db) : db(db) {} };
// Test:
MockDatabase mockDb;
mockDb.expectInsert(testUser);
UserService svc(mockDb);     // inject mock
svc.save(testUser);
mockDb.verify();              // check expectations met
```

6.20 CQRS — Command Query Responsibility Segregation

CQRS

CQRS separates the **command** model (writes: create, update, delete) from the **query** model (reads).
Why? In complex domains, the data model optimised for writing (normalised, relational) differs from the model optimised for reading (denormalised, pre-joined, pre-aggregated for the UI). Trying to use one model for both leads to complex queries and impedance mismatch.
Command side: receives commands (`PlaceOrder`, `CancelOrder`), validates them, updates the domain model, persists to the write store.
Query side: separate read model (possibly a different database), updated asynchronously from the write side (often via events). Queries hit the read model directly — simple, fast, no joins needed.

CQRS Data Flow

```
Client
  |--- Command (PlaceOrder) ---> Command Handler ---> Write DB
  |                                    |
  |                              Domain Event (OrderPlaced)
  |                                    |
  |                             Event Handler ---> Read DB (denormalised)
  |
  |--- Query (GetOrderHistory) ---> Query Handler ---> Read DB
```

6.21 Event Sourcing

Event Sourcing

Instead of storing the **current state** of an entity, store the **sequence of events** that led to that state. The current state is derived by replaying all events.
Example: A bank account doesn't store `balance = 500`. It stores:
1. `AccountOpened{initial=0}`
2. `MoneyDeposited{amount=1000}`
3. `MoneyWithdrawn{amount=500}`

Replaying these gives balance = 500. **Every state the account ever had is preserved.**

Event Sourcing Benefits

- **Complete audit log**: every change is recorded with who, when, and what. Invaluable for compliance (banking, healthcare, legal).
- **Time travel**: reconstruct state at any point in history.
- **Event-driven integration**: publish events to other systems. No polling, no two-phase-commit needed.
- **Retroactive corrections**: replay events with a bug fix applied.
- **Multiple read models**: derive different views from the same event stream.

Challenges: querying current state requires replay (mitigated by **snapshots** — periodically persist the derived state and only replay events after the snapshot).

6.22 Hexagonal Architecture (Ports and Adapters)

Hexagonal Architecture (Alistair Cockburn, 2005)

The core **domain** (business logic) sits in the centre, surrounded by **ports** (interfaces) and **adapters** (implementations of those interfaces).
- **Primary/Driving ports**: entry points to the domain (REST API, CLI, message queue consumer). Adapters: HTTP controller, CLI parser.
- **Secondary/Driven ports**: the domain calls out to these (database, external services). Adapters: SQL repository, Redis cache, HTTP client.

Key rule: the domain knows nothing about the outside world. It only depends on interfaces (ports). Adapters are swappable without changing the domain.
Result: the domain can be tested in complete isolation (inject mock adapters). Switching from PostgreSQL to MongoDB requires only a new adapter.

Hexagonal Architecture Layers

```
[HTTP Controller] ---> (Primary Port: OrderService interface)
                                    |
                        [Domain: Order, OrderService, etc.]
                                    |
                          (Secondary Port: OrderRepository interface)
                                    |
                    [SQL Adapter] or [InMemory Adapter (for tests)]
```

6.23 Chain of Responsibility Pattern

Chain of Responsibility

Intent: Pass a request along a chain of handlers. Each handler decides either to process the request or pass it to the next handler.
Use when: More than one handler can process a request, and the correct handler is determined at runtime. Decouples senders from receivers.
Real-world examples: middleware pipelines (Express.js, ASP.NET, Django middleware), event bubbling in UIs, logging frameworks (log level filters), ATM cash dispensing.

```
class Handler {
    Handler* next = nullptr;
public:
    Handler* setNext(Handler* h) { next = h; return h; }
    virtual void handle(int request) {
        if (next) next->handle(request);
    }
};

class AuthHandler : public Handler {
    void handle(int req) override {
        if (!isAuthenticated(req)) { respond(401); return; }
        Handler::handle(req);  // pass to next
    }
};
class RateLimitHandler : public Handler {
    void handle(int req) override {
        if (isRateLimited(req)) { respond(429); return; }
        Handler::handle(req);
    }
};
class BusinessHandler : public Handler {
    void handle(int req) override { processRequest(req); }
};

// Build chain
AuthHandler auth;
RateLimitHandler rate;
BusinessHandler biz;
auth.setNext(&rate)->setNext(&biz);
auth.handle(request);  // goes through auth -> rateLimit -> business
```

Chapter 6 Additional — Key Takeaways

- **SOLID**: SRP (one reason to change), OCP (extend don't modify), LSP (subtypes must be substitutable), ISP (small interfaces), DIP (depend on abstractions).
- **Dependency Injection**: inject dependencies from outside; enables mocking in tests. Prefer constructor injection.
- **CQRS**: separate read and write models. Enables independent scaling and different optimisations for reads vs. writes.
- **Event Sourcing**: store events, not state. Complete audit trail, time travel, retroactive fixes. Use snapshots for query performance.
- **Hexagonal Architecture**: domain in centre, ports as interfaces, adapters as swappable implementations. Business logic is infrastructure-free.
- **Chain of Responsibility**: middleware pipelines; each handler decides to process or pass on the request.

Chapter 7

CAP Theorem & Distributed Systems

When a single machine isn't enough — for reliability, capacity, or geographic distribution — you build distributed systems. But distribution introduces a new class of problems: partial failures, network delays, and inconsistent views of data. This chapter covers the fundamental theory and trade-offs.

7.1 What Makes Distributed Systems Hard

The Challenges of Distribution

In a single-process system, shared memory is consistent and operations are atomic. In a distributed system:

- **Partial failures**: individual nodes or network links can fail while others continue. A server receiving no message can't tell if the sender died, the network is down, or the message is just slow.
- **No shared clock**: nodes have independent clocks that drift. There is no global "now" — clocks must be synchronised (NTP), and even then drift is non-zero.
- **Unbounded message delay**: a message may arrive instantly, be delayed seconds, arrive out of order, be duplicated, or never arrive.
- **No global state**: nodes see different snapshots of the system.

The Two Generals Problem

Two armies (nodes) must coordinate an attack over an unreliable messenger (network). Army 1 sends: "Attack at dawn." Before committing, Army 1 needs confirmation. Army 2 sends the confirmation — but how does Army 2 know the confirmation arrived? It must wait for an ACK of the ACK — which itself can be lost.
Proven impossibility: with unreliable message delivery, two nodes can never achieve 100% guaranteed common knowledge through finite message exchange. This is the fundamental reason that distributed consensus is hard.

7.2 The CAP Theorem

CAP Theorem (Brewer, 2000)

A distributed data store can satisfy **at most two** of these three properties simultaneously:

- **Consistency (C)** — every read receives the *most recent write* or an error. All nodes see the same data at the same time. (Note: this is *linearisability*, not ACID consistency.)
- **Availability (A)** — every request receives a (non-error) response, though it may not be the most recent write.
- **Partition Tolerance (P)** — the system continues operating even when an arbitrary number of messages between nodes are dropped (network partition).

Since network partitions are a physical reality in any distributed system, **P is not optional**. The real trade-off is always **C vs. A during a partition**.

CAP Choices in Practice

Choice	Description	Examples
CP (Consistency + Partition)	Returns error if can't guarantee consistency	HBase, ZooKeeper, etcd
AP (Availability + Partition)	Returns possibly stale data, always responds	Cassandra, CouchDB, DynamoDB

No system chooses CA (Consistency + Availability without Partition tolerance) — that's only possible in a single-node system that cannot be partitioned.

CAP is Not Binary

CAP is often misused. In practice:

- Partitions are *rare*. Between partitions, you can have both C and A.
- Real systems tune consistency and availability independently per operation.
- CAP's definition of "consistency" (linearisability) is very strong; many useful weaker forms exist.

The **PACELC** model (next section) is more nuanced.

7.3 Consistency Models

Not all consistency is equal. Weaker models allow higher performance and availability.

Consistency Model Spectrum (strongest to weakest)

1. **Linearisability (Strong Consistency)** — operations appear to happen instantaneously at some point between invocation and response. The globally "correct" view. Expensive: requires coordination on every read/write.
2. **Sequential Consistency** — all operations appear in some sequential order that is consistent for all processes (not necessarily real-time order).
3. **Causal Consistency** — operations that are causally related (A's write causes B's read) are seen in order by all nodes. Unrelated operations may be seen in different orders.
4. **Read-Your-Writes** — a process always sees its own writes when it subsequently reads. Others may not see them yet.
5. **Eventual Consistency** — if no new updates are made, all replicas will *eventually* converge to the same value. No timing guarantee. Most Amazon DynamoDB defaults to this.

Eventual Consistency in DNS

When you update a DNS record, the change propagates across the world's DNS servers. For minutes or hours, some servers return the old IP, others return the new one. Eventually (when TTL expires and caches refresh), all servers agree. This is eventual consistency in a widely deployed, decades-old distributed system.

7.4 PACELC Theorem

PACELC (Daniel Abadi, 2012)

PACELC extends CAP to also describe the trade-off during *normal operation* (no partition):

If Partition: choose between Availability or Consistency
Else (no partition): choose between Latency or Consistency

Notation: PA/EL (prefer availability during partitions, prefer low latency normally) vs. PC/EC (prefer consistency in both scenarios).

Examples:

- **Cassandra**: PA/EL — available during partitions, low latency normally (tunable consistency).
- **DynamoDB**: PA/EL — similar.
- **BigTable / HBase**: PC/EC — consistent always, higher latency.
- **etcd / ZooKeeper**: PC/EC — strongly consistent at the cost of latency.

7.5 ACID vs. BASE

ACID (Traditional Databases)

- **Atomicity** — a transaction either fully commits or fully rolls back. No partial state is visible.
- **Consistency** — a transaction takes the database from one valid state to another (application-defined invariants are maintained).
- **Isolation** — concurrent transactions are isolated from each other; the result is as if they ran serially.
- **Durability** — once committed, a transaction's effects survive crashes. Written to disk.

Used by: PostgreSQL, MySQL, Oracle, SQLite. Strong guarantees but harder to scale.

BASE (NoSQL / Distributed Systems)

- **Basically Available** — the system is available most of the time, even if some responses may be stale or incorrect.
- **Soft state** — the state may change over time even without input (as replicas sync up).
- **Eventually Consistent** — the system will reach consistency eventually.

BASE trades correctness for availability and performance. Used by: Cassandra, DynamoDB, MongoDB (default mode), CouchDB.

Choosing ACID vs. BASE

- **ACID**: financial transactions, inventory (you can't oversell), any domain where wrong data causes real harm.
- **BASE**: social media feeds, shopping carts (temporary inconsistency is acceptable), analytics, caching layers, search indexes.
- Many modern systems use ACID for critical data and BASE for derived/cached data.

7.6 Eventual Consistency in Practice

Conflict Resolution

When the same data is updated concurrently on two replicas (during a partition), there's a conflict. Strategies:

- **Last Write Wins (LWW)**: the write with the highest timestamp wins. Simple, but clock skew can cause correct writes to be discarded.
- **Multi-Version Concurrency Control (MVCC)**: keep all versions; return them all to the client, which resolves the conflict (Amazon's shopping cart paper does this — show all items from all cart versions).
- **CRDTs** (Conflict-free Replicated Data Types): data structures designed to merge automatically without conflicts. A counter CRDT merges by summing per-replica increments; a set CRDT merges by union. Used in Redis, Riak, collaborative editing (Google Docs uses an OT variant).

7.7 Distributed Consensus

The Consensus Problem

Given a set of nodes that can fail, get them to **agree on a single value** despite failures. Requirements:

- **Agreement**: all non-faulty nodes decide the same value.
- **Validity**: the decided value was proposed by some node.
- **Termination**: every non-faulty node eventually decides.

The **FLP impossibility theorem** (Fischer, Lynch, Paterson, 1985) proves that in an asynchronous system with even one possible crash, no deterministic algorithm can guarantee all three properties. Real protocols work around this with timeouts and probabilistic assumptions.

Paxos (Lamport, 1989): The foundational consensus algorithm. A leader (proposer) coordinates a two-phase protocol to get a majority (quorum) of acceptors to agree on a value. Notoriously difficult to understand and implement correctly.

Raft (Ongaro & Ousterhout, 2014): Designed to be *understandable*. Separates consensus into leader election, log replication, and safety.

- One node is the **leader**. Clients send writes to the leader.
- The leader replicates log entries to followers.
- Once a majority have written an entry, it is **committed**.
- If the leader fails, followers elect a new leader via random timeouts.

Used by: etcd (Kubernetes uses etcd), CockroachDB, TiKV, Consul.

7.8 Vector Clocks and Logical Clocks

Lamport Timestamps

In distributed systems, there is no global clock. **Lamport timestamps** provide **logical time**: each node maintains a counter. Before sending a message, increment and include the counter. On receiving, set local counter to $\max(\text{local}, \text{received}) + 1$.
Provides **happens-before** ordering: if A happened before B, then $\text{ts}(A) < \text{ts}(B)$. But not vice versa (concurrent events can have any timestamps).

Vector Clocks

Extend Lamport timestamps to track causality precisely. Each node i maintains a vector V of length N (number of nodes). $V[i]$ is node i's own event count. When sending a message, include the full vector.
A causally precedes B ($A \to B$) if and only if $V_A[j] \leq V_B[j]$ for all j, and at least one is strictly less.
If neither $A \to B$ nor $B \to A$, events are **concurrent** (happened independently). Used by: Amazon DynamoDB, Riak, distributed version control (Git's merge algorithm is similar).

7.9 Two-Phase Commit (2PC)

Two-Phase Commit

2PC is a protocol to achieve **atomic distributed transactions** (all nodes commit or all abort):
Phase 1 — Prepare:
1. The **coordinator** sends a PREPARE message to all **participants**.
2. Each participant writes its changes to a log, locks resources, and replies YES (ready to commit) or NO (abort).

Phase 2 — Commit/Abort:
1. If *all* replied YES: coordinator sends COMMIT to all.
2. If *any* replied NO: coordinator sends ABORT to all.
3. Participants commit or abort and release locks.

2PC Failure Modes

- **Coordinator failure after prepare but before commit**: participants have voted YES and locked resources, but don't know whether to commit or abort. They are **blocked** (holding locks) until the coordinator recovers. This is the fundamental weakness of 2PC.
- **Network partition during commit**: some participants commit, others abort — **split brain**.

3PC (Three-Phase Commit) adds an extra phase to eliminate blocking, but is rarely used in practice due to complexity and sensitivity to network assumptions. Most systems use Raft/Paxos-based approaches or saga patterns for distributed transactions.

7.10 The Saga Pattern

Sagas

Instead of a distributed transaction (2PC), a **saga** is a sequence of local transactions, each publishing an event that triggers the next step. If a step fails, **compensating transactions** undo prior steps.
Example (e-commerce order):
1. Reserve inventory.
2. Charge credit card.
3. Update order status.

If step 2 fails: run compensating transaction to release the inventory reservation. No distributed locks, no 2PC. Each service owns its own data. Eventual consistency between services.

Sagas are widely used in microservices (no distributed transactions across services). Two implementations: **choreography** (events trigger next steps, no central coordinator) and **orchestration** (a saga orchestrator explicitly calls each step).

Chapter 7 — Key Takeaways

- Distributed systems face **partial failures**, **no global clock**, and **unreliable networks** — these are unavoidable.
- **CAP**: you must choose between Consistency and Availability during a network partition. Partition tolerance is not optional.
- **PACELC** adds the latency vs. consistency trade-off during normal operation.
- **ACID** = strong guarantees, harder to scale. **BASE** = flexible, eventually consistent, scales better.
- **Raft** is the practical consensus algorithm: leader election + log replication.
- **Vector clocks** distinguish causally related from concurrent events.
- **2PC** achieves atomic distributed transactions but blocks on coordinator failure. **Sagas** are the modern microservices alternative.

7.11 Paxos: Step by Step

Paxos Roles

- **Proposer**: proposes values to be agreed upon.
- **Acceptor**: votes to accept or reject proposals.
- **Learner**: learns the chosen value once consensus is reached.

A quorum is any majority of acceptors ($\lfloor N/2 \rfloor + 1$ out of N). Two quorums always overlap by at least one acceptor — this is what makes Paxos safe.

Paxos Phase 1 — Prepare

1. Proposer chooses a unique proposal number n (monotonically increasing, often a (timestamp, server-id) tuple).
2. Proposer sends **Prepare(n)** to all acceptors.
3. Each acceptor responds with a **Promise**:
   - Promises to never accept proposals numbered $< n$.
   - Returns the highest-numbered proposal it has already accepted (if any), along with its value.

   If the acceptor has already promised a higher number $n' > n$, it ignores this prepare (or responds with a NACK).

If the proposer receives promises from a majority, it proceeds to Phase 2.

Paxos Phase 2 — Accept

1. The proposer selects the value:
   - If any acceptor returned a previously accepted value, the proposer **must** use the value from the highest-numbered accepted proposal.
   - If no acceptor returned any prior accepted value, the proposer is free to choose its own value.
2. Proposer sends **Accept**(n, v) to all acceptors.
3. Each acceptor accepts the proposal if it has not promised a higher number. It records the acceptance and notifies learners.
4. If a quorum of acceptors accept the same (number, value), the value is **chosen**.

Paxos Safety Intuition

Paxos can **never** choose two different values (safety). Why? If value v was chosen in round n (accepted by a quorum Q_1), then in any future round $n' > n$, the proposer's Phase 1 quorum Q_2 must overlap Q_1 (both are majorities). The proposer will see that some acceptor in $Q_1 \cap Q_2$ already accepted v, so it must propose v again. The same value keeps being re-proposed.

Duelling Proposers (Livelock)

If two proposers repeatedly try higher proposal numbers, interrupting each other's Phase 2 with higher Phase 1 prepares, no value is ever chosen. **Multi-Paxos** solves this by electing a stable **leader** (distinguished proposer) that runs repeated rounds of Phase 2 without repeating Phase 1. Raft makes this the core of its design.

7.12 Raft: Step by Step

Raft Terms and Leader Election

Time is divided into **terms** (monotonically increasing integers). Each term begins with a **leader election**. There is at most one leader per term.

Election:

1. Each server starts as a **follower**. It starts a random election timeout (150–300 ms). If it doesn't hear from a leader before timeout, it becomes a **candidate**.
2. Candidate increments its term, votes for itself, sends **RequestVote** RPCs to all others.
3. A server grants its vote if: (a) it hasn't voted in this term, and (b) the candidate's log is at least as up-to-date as the voter's log.
4. If a candidate receives votes from a majority: it becomes **leader**.
5. If no majority before timeout: new election with higher term.

Raft Log Replication

Once elected, the leader handles all client requests:

1. Leader appends the command to its log as a new entry (with current term number).
2. Leader sends **AppendEntries** RPCs to all followers in parallel.
3. Once a majority of servers have written the entry to their logs, the entry is **committed**. The leader applies it to its state machine and responds to the client.
4. On the next **AppendEntries** (or heartbeat), the leader informs followers of the commit index; they apply committed entries to their state machines.

Log Matching Property: If two logs have an entry with the same index and term, all entries up to that point are identical. Enforced by the consistency check in **AppendEntries**: leaders include the index and term of the entry preceding the new entries; followers reject if their log doesn't match.

Raft Safety: Election Restriction

Raft guarantees a leader always has all committed entries. A candidate cannot win an election unless its log is at least as up-to-date as a majority of the cluster. "More up-to-date" means: higher last log term, or same term but longer log. This ensures that when a leader is elected, it already has all committed entries and doesn't need to fetch them from followers.

7.13 Gossip Protocols

Gossip (Epidemic) Protocol

Gossip protocols spread information through a network the same way rumours spread through people: each node periodically selects a few random neighbours and exchanges state. Information reaches the whole cluster in $O(\log N)$ rounds.

Properties:

- **Decentralised**: no coordinator. Every node is equal.
- **Fault-tolerant**: nodes failing doesn't stop propagation.
- **Eventual convergence**: after enough rounds, all nodes agree.
- **Scalable**: each node only talks to a few others per round, so total messages per round = $O(k \cdot N)$ (constant factor k).

Used by: Cassandra (node failure detection and ring membership via Gossiper), Redis Cluster (cluster state gossip), Consul (health checks and service discovery), Amazon S3 (metadata consistency), Bitcoin (transaction propagation).

Phi Accrual Failure Detector

Used by Cassandra's gossip. Instead of a binary "up/down" verdict, it outputs a continuously growing **suspicion level** ϕ. When the last heartbeat was very recent, ϕ is low. As time passes without a heartbeat, ϕ rises. At $\phi = 8$, we're 99.97% confident the node has failed.

Advantage over timeout-based detectors: adaptive to actual heartbeat latency (no need to tune timeouts manually across different network environments).

7.14 Bloom Filters

Bloom Filter

A **Bloom filter** is a probabilistic data structure that answers "is element x in the set?" in $O(1)$ with no false negatives but a configurable false positive rate.

Structure: a bit array of m bits + k independent hash functions.

Insertion: hash x with all k functions, set those k bits to 1.

Query: hash x with all k functions. If any of the k bits is 0, x is **definitely not** in the set. If all are 1, x is **probably in** the set (false positive possible — another element may have set those bits).

False positive rate: $\approx (1 - e^{-kn/m})^k$ where n = number of insertions. Configurable by choosing m and k.

Bloom Filter Use Cases

- **Database query optimisation**: before checking disk for a key, check a Bloom filter. If negative: definitely not there (skip disk I/O). If positive: check disk. Saves I/O for non-existent keys. Used by: LevelDB, RocksDB, Cassandra, HBase (one per SSTable).
- **Cache penetration defence**: cache a Bloom filter of all valid user IDs. Requests for non-existent IDs are rejected at the filter, never hitting the database.
- **Web crawler deduplication**: has this URL been crawled? ≈0% false negatives mean we never re-crawl. Small false positive rate means we occasionally skip a valid URL — acceptable.
- **Malware detection**: Chrome's Safe Browsing downloads a Bloom filter of malicious URLs. Local check is instant; server query only on positive.

7.15 Merkle Trees

Merkle Tree

A **Merkle tree** is a binary tree where every leaf node contains the hash of a data block, and every internal node contains the hash of its children's hashes. The root hash (Merkle root) is a **cryptographic commitment** to the entire data set.
Efficient verification: to prove a single leaf is in the tree, you only need $O(\log N)$ hashes (the sibling hashes along the path from leaf to root) — not the entire dataset.
Efficient diff: to find which parts of a large dataset differ between two nodes, compare their Merkle roots. If roots match, data is identical. If not, recurse into the subtree where hashes diverge — efficiently finding the differing blocks.

Merkle Trees in Practice

- **Bitcoin**: the Merkle root of all transactions in a block is stored in the block header. Lightweight clients verify transactions with $O(\log N)$ hashes instead of downloading all transactions.
- **Git**: every commit contains the Merkle root (tree hash) of the entire repository state. Efficient diff, integrity verification.
- **Cassandra anti-entropy**: each node builds a Merkle tree of its data. Nodes exchange trees to find divergent ranges and trigger repair.
- **IPFS / distributed storage**: content-addressed with hashes; Merkle trees enable efficient partial verification.

7.16 Byzantine Fault Tolerance

Byzantine Faults

In our earlier discussion, we assumed **crash-fail** faults: a failed node stops responding. **Byzantine faults** are worse: a node can behave *arbitrarily* — sending different values to different nodes, lying about its state, colluding with other faulty nodes. Named after the "Byzantine Generals Problem" (Lamport, 1982).
The result: with crash faults, you need $2f + 1$ nodes to tolerate f failures. With Byzantine faults, you need $\mathbf{3f + 1}$ nodes to tolerate f Byzantine failures.
PBFT (Practical Byzantine Fault Tolerance, Castro & Liskov, 1999) was the first practical BFT protocol. BFT protocols are used in:

- **Blockchains**: Tendermint (Cosmos), HotStuff (Facebook's Libra/Diem), Ethereum's Proof-of-Stake consensus.
- **Aircraft avionics**: redundant flight computers with BFT voting.
- **Space systems**: interplanetary systems where cosmic rays can cause bit flips (simulating Byzantine behaviour).

Chapter 7 Additional — Key Takeaways

- **Paxos**: two phases — Prepare (promise round) and Accept (commit round). Quorum overlap guarantees only one value is ever chosen. Duelling proposers cause livelock; Multi-Paxos uses a stable leader.
- **Raft**: leader election with random timeouts, log replication with majority acknowledgement. Election restriction ensures leaders have all committed entries.
- **Gossip**: $O(\log N)$ convergence, decentralised, fault-tolerant. Cassandra, Consul, Redis Cluster all use it.
- **Bloom filters**: $O(1)$ membership queries with no false negatives. Used by every major LSM-tree database to avoid disk I/O for missing keys.
- **Merkle trees**: $O(\log N)$ proof of membership and efficient set diff. Foundation of Git, Bitcoin, and distributed anti-entropy repair.
- **Byzantine faults**: arbitrary misbehaviour. Requires $3f + 1$ nodes. Core of blockchain consensus protocols.

Chapter 8

Scaling Technologies

How do you go from serving 100 users to 10 million? Not by magic — by understanding which part of your system is the bottleneck, and applying the right technique. This chapter is a tour of the scaling toolkit used by every major internet company.

8.1 Vertical vs. Horizontal Scaling

Vertical Scaling (Scale Up)

Add more resources to a **single machine**: more CPU cores, more RAM, faster disk, better network card.
Pros:
- No application changes needed (no distributed systems complexity).
- Low latency (all components on one machine, no network hops).

Cons:
- Hard physical limits: a single AWS instance tops out at 224 vCPUs, 24 TB RAM.
- Single point of failure: one machine goes down, everything goes down.
- Cost grows super-linearly (the biggest machines cost much more per unit).

Horizontal Scaling (Scale Out)

Add more machines and distribute the load across them. Each machine handles a fraction of the total workload.
Pros:
- Near-linear scalability (in theory): $2\times$ machines $\approx 2\times$ capacity.
- Fault tolerance: if one machine fails, others continue.
- Commodity hardware: cheap machines instead of expensive ones.

Cons:
- Application must be designed for distribution: state management, shared data, network latency between nodes.
- Operational complexity: orchestration (Kubernetes), service discovery, monitoring.

Most modern systems combine both: scale vertically until diminishing returns, then scale horizontally.

8.2 Load Balancing

Load Balancer

A **load balancer** sits in front of a pool of servers and distributes incoming requests across them. It:

- Spreads load evenly, preventing any one server from being overwhelmed.
- Performs **health checks** and routes around failed servers.
- Provides a single IP/DNS endpoint to clients (hides the server pool).
- Can terminate SSL (handling encryption, offloading CPU from servers).

Types: **L4** (TCP/UDP layer, routes by IP+port), **L7** (HTTP layer, routes by URL, headers, cookies). L7 is more flexible but has higher overhead.

8.3 Load Balancing Algorithms

Common Load Balancing Algorithms

- **Round Robin**: each request goes to the next server in a circular list. Simple, works well when requests have similar cost. Used by nginx (default).
- **Weighted Round Robin**: servers have weights; higher-capacity servers get proportionally more requests.
- **Least Connections**: route to the server with the fewest active connections. Better than round robin when requests have variable duration (long-polling, streaming).
- **IP Hash / Sticky Sessions**: hash the client's IP to always route them to the same server. Needed when server-side sessions hold client state (though better to externalise state to Redis).
- **Random with two choices (Power of Two)**: pick two servers at random, send to the less loaded. Surprisingly close to optimal with $O(1)$ overhead.

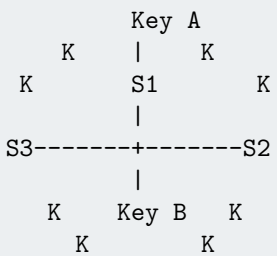
8.4 Consistent Hashing

When the number of servers changes (scaling up/down, server failure), a naive hash approach (`server = key % N`) reassigns almost all keys to different servers — cache invalidation tsunami.

Consistent Hashing

Place both servers and keys on a **ring** (imaginary circle, values 0–2^{32}). To find a key's server: hash the key to a point on the ring; walk clockwise to the first server.
Adding a server: only the keys between the new server and its predecessor on the ring are remapped. All other keys are unaffected.
Removing a server: only its keys move to the next server clockwise.
With K keys and N servers: a server change affects only K/N keys, not all K.
Virtual nodes: assign each physical server multiple points on the ring to improve balance. Used by: Cassandra, DynamoDB, Redis Cluster, Memcached (ketama).

Consistent Hashing Ring (3 servers, 12 keys)



Keys between S3 and S1 (going clockwise) are served by S1. Adding a new server between S1 and S2 only reassigns keys that fall between it and S1.

8.5 Caching Strategies

Caching is the single most effective scaling technique. The goal: serve frequent reads from fast storage (memory) instead of slow storage (database, disk).

Cache Placement Strategies

- **Client-side cache**: browser cache (HTTP Cache-Control headers), in-process cache (LRU cache in your app). Zero network overhead on hit.
- **CDN (Content Delivery Network)**: geographically distributed caches close to users. Serves static content (images, JS, CSS, videos) from the nearest edge node.
- **Reverse proxy cache** (Varnish, nginx): caches HTTP responses in front of your servers. Entirely transparent to the app.
- **Application-level cache** (Redis, Memcached): your app explicitly caches computed results, DB query results, session data.
- **Database query cache**: the DB itself caches recent query results. (MySQL's query cache was removed in 8.0 due to scalability issues.)

Cache Eviction Policies

- **LRU (Least Recently Used)**: evict the item not accessed for the longest time. Good general-purpose policy.
- **LFU (Least Frequently Used)**: evict the least often accessed item. Better for workloads with a stable hot set.
- **FIFO**: evict the oldest item. Simple, but can evict hot items.
- **TTL (Time-To-Live)**: evict after a fixed time, regardless of access pattern. Controls staleness.

Redis defaults to LRU with a TTL option. Memcached uses LRU.

8.6 Cache Invalidation

"There are only two hard things in Computer Science: cache invalidation and naming things." — Phil Karlton

Cache Update Patterns

- **Cache-Aside (Lazy Loading)**: the application manages the cache. On a miss: read from DB, store in cache, return to client. On a write: update DB, **invalidate** the cache entry (or update it). Simple, flexible. Cache only contains what's been requested.
- **Write-Through**: every write goes to both the cache and the DB synchronously. Cache is always consistent. Writes are slower (two writes).
- **Write-Behind (Write-Back)**: writes go to cache only; the cache asynchronously writes to DB later. Fast writes, but risk of data loss if the cache dies before flushing.
- **Read-Through**: cache sits in front; on a miss, the cache fetches from DB itself (not the application).

Cache Pitfalls

- **Cache stampede (thundering herd)**: cache entry expires; simultaneously, thousands of requests miss and all query the DB. Fix: mutex on cache miss (only one thread fetches), or probabilistic early recomputation.
- **Cache penetration**: requests for non-existent keys always miss the cache and hit the DB. Fix: cache negative results ("null sentinel") or use a Bloom filter.
- **Cache avalanche**: many keys expire at the same time, overwhelming the DB. Fix: randomise TTL values (add random jitter).

8.7 Content Delivery Networks (CDN)

CDN

A **CDN** is a globally distributed network of **edge servers** (Points of Presence, PoPs) close to end users. Static assets (images, videos, CSS, JS bundles) are cached at the edge. When a user requests a file:

1. DNS resolves to the nearest edge server (via Anycast or geographic routing).
2. Edge serves from cache (if hit): latency is the distance to the edge, not your origin datacenter.
3. On cache miss: edge fetches from origin, caches it, serves the user.

Examples: Cloudflare, AWS CloudFront, Akamai, Fastly. Cache control: via HTTP `Cache-Control: max-age=86400` header.

CDNs don't just serve static content. Modern CDNs offer:

- **Edge compute** (Cloudflare Workers, Lambda@Edge): run code at the edge.
- **DDoS protection**: absorb volumetric attacks at the edge before they reach your origin.
- **WAF (Web Application Firewall)**: filter malicious HTTP requests.

8.8 Database Scaling: Read Replicas

A single database write leader is a common bottleneck. Most applications are read-heavy (read:write ratio of 10:1 or higher).

Read Replicas

Create one or more **replica databases** that continuously sync from the **primary (leader)**. Route all writes to the primary; distribute reads across replicas.

Replication types:

- **Synchronous**: primary waits for replica to acknowledge before confirming the write. Strong consistency, higher write latency.
- **Asynchronous** (most common): primary commits immediately; replica catches up asynchronously. Lower write latency, but replicas may be slightly stale (**replication lag**).

Replication lag: replicas are always slightly behind the primary. Don't use replicas for reads that must reflect the most recent write (e.g., "show the item I just purchased"); use the primary for that.

8.9 Database Sharding

When even the primary DB can't handle write load or data volume, shard.

Sharding (Horizontal Partitioning)

Split the database into **shards**, each holding a subset of the data. Each shard is an independent database handling reads and writes for its portion.

Sharding strategies:

- **Range-based**: shard by key range (e.g., users A–M on shard 1, N–Z on shard 2). Simple, allows range queries. Risk of **hotspots** (all new users go to one shard).
- **Hash-based**: hash the shard key; assign to shard by hash mod N. Even distribution. No range queries across shards. Add consistent hashing to avoid full reshuffling on shard addition.
- **Geographic**: users in Europe on EU shards (data sovereignty), US users on US shards.

Sharding Complications

- **Cross-shard queries** (joins, aggregations across shards) are expensive or impossible. Denormalise data to avoid them.
- **Cross-shard transactions**: require 2PC or sagas.
- **Re-sharding**: adding shards requires migrating data. Use consistent hashing to minimise movement.
- **Hotspot**: if one shard gets disproportionate traffic (celebrity problem: one user's data is accessed by millions), it becomes a bottleneck. Use **cell architecture** or dedicated shards for hot keys.

8.10 Message Queues and Event-Driven Architecture

Message Queue

A **message queue** decouples producers (who send messages) from consumers (who process them). Producers publish messages to a queue; consumers read at their own pace.
Benefits:

- **Asynchrony**: producers don't wait for consumers. Reduces latency for the producer.
- **Load levelling**: absorbs traffic spikes. The queue buffers bursts; consumers process at a steady rate.
- **Decoupling**: producer and consumer can be deployed independently.
- **Retry**: if a consumer fails processing, the message can be requeued.

Apache Kafka

Kafka is a **distributed log** (not a traditional queue):

- Messages are written to an append-only **log** (topic + partition).
- Messages are retained for a configurable period (days/weeks/forever).
- **Multiple consumer groups** can read the same messages independently (replaying the log) — unlike traditional queues where a message is consumed once.
- Partitions enable horizontal scaling of both producers and consumers.
- Used for: event streaming, activity tracking, metrics, log aggregation, stream processing (Kafka Streams, Apache Flink).

8.11 Microservices vs. Monolith

Monolith

A **monolith** is a single deployable unit containing all functionality. All code runs in one process; communication between components is function calls.
Pros: Simple to develop, test, and deploy. Fast inter-component calls (no network). Easy to debug (one process). Good for small teams and early-stage products.
Cons: As it grows: long build times, large teams stepping on each other, a single bad deploy takes everything down. Hard to scale individual components.

Microservices

Each business capability is a **separate service** with its own codebase, data store, and deployment pipeline. Services communicate over the network (REST, gRPC).
Pros:

- Independent deployment: deploy service A without affecting service B.
- Independent scaling: scale only the checkout service, not the entire app.
- Technology heterogeneity: each service can use the best language/framework.
- Fault isolation: one service crashing doesn't take down others.

Cons:

- Network latency between services (function call: ns; network call: ms).
- Distributed tracing and debugging is complex.
- Data consistency across services (no easy ACID transactions).
- Operational overhead: service mesh, service discovery, health checks, many repos.

The Strangler Fig Pattern

Don't rewrite a monolith from scratch (risky). Instead, incrementally extract services: route specific routes/features to new services while the monolith handles the rest. Over time, the monolith "dies" as its functionality is migrated. Used by Netflix, Shopify, Amazon.

8.12 API Gateways

API Gateway

An **API gateway** is a single entry point for all client requests. It:
- **Routes** requests to the appropriate microservice.
- **Aggregates** multiple service calls into one response (BFF pattern).
- **Authenticates** requests (validate JWT, OAuth tokens) centrally.
- **Rate limits** clients.
- **Caches** responses.
- Handles **SSL termination**, request logging, and metrics.

Examples: AWS API Gateway, Kong, Envoy, Nginx. The alternative (clients calling services directly) exposes internal topology and requires each service to handle auth/rate-limiting individually.

8.13 Rate Limiting

Rate Limiting

Rate limiting caps the number of requests a client can make in a time window. Protects against:
- **Abuse and DDoS**: a single client can't overwhelm the system.
- **Fair usage**: prevents one tenant from monopolising shared resources.

Algorithms:
- **Token Bucket**: bucket holds up to N tokens. Each request consumes one token. Tokens refill at rate r/sec. Allows bursts up to N.
- **Leaky Bucket**: requests enter a queue (bucket); they drain at a fixed rate. Enforces a smooth, constant output rate. Excess requests are dropped.
- **Fixed Window Counter**: count requests per time window (e.g., per minute). Simple, but the boundary between windows can double the allowed rate.
- **Sliding Window Log**: log timestamps of each request; count those within the window. Accurate but memory-intensive.
- **Sliding Window Counter**: approximate using counts from the current and previous window. Good balance of accuracy and memory.

8.14 Circuit Breakers

Circuit Breaker Pattern

Named after the electrical circuit breaker, this pattern **detects failures** and **stops propagating them**. The circuit breaker wraps calls to an external service and has three states:
- **Closed** (normal): requests pass through. Monitor failure rate.
- **Open** (tripped): the downstream service is failing (failure rate exceeded threshold). All requests fail immediately without trying the service. The caller gets a fast error (or a fallback), not a slow timeout.
- **Half-Open**: after a cooldown period, allow a probe request. If it succeeds: close the circuit. If it fails: open again.

Without a circuit breaker, if Service A calls Service B and B is slow/down: A's threads pile up waiting for B's timeout (∼30s). A runs out of threads. A itself becomes unavailable. This is **cascading failure**.

A circuit breaker breaks the cascade: A immediately returns an error or cached response instead of waiting, and stops hammering the failing B.

Libraries: Netflix Hystrix (deprecated), Resilience4j (Java), go-circuitbreaker (Go).

8.15 Service Discovery

Service Discovery

In a dynamic environment (containers, auto-scaling), server IPs change constantly. **Service discovery** lets services find each other without hardcoded IPs.
Client-side discovery: the client queries a **service registry** (e.g., etcd, Consul, Eureka) for the IP list of a service, then picks one (with client-side load balancing).
Server-side discovery: the client sends requests to a load balancer; the load balancer queries the registry and routes to an instance. Client is unaware of the registry.
Kubernetes uses DNS-based discovery: each Service gets a stable DNS name (`my-service.my-namespace.svc.cluster.local`); kube-proxy load-balances via iptables/IPVS rules.

8.16 The Full Stack at a Glance

Scaling Architecture of a Typical Web Service

```
User
  |
  v
CDN (static assets)
  |
  v
DNS (GeoDNS -> nearest region)
  |
  v
Load Balancer (L7, HTTPS termination)
  |
  +---> App Servers (stateless, horizontally scaled)
          |           |
          v           v
        Cache       Message Queue
       (Redis)       (Kafka)
          |             |
          v             v
     Primary DB     Worker Pool
   (write leader)  (async processing)
      |
      +---> Read Replicas (distribute reads)
```

Chapter 8 — Key Takeaways

- **Vertical scaling** is simple but has limits. **Horizontal scaling** scales further but requires distributed systems design.
- **Load balancers** distribute traffic; **consistent hashing** keeps cache assignment stable as servers change.
- **Caching** is the highest-leverage scaling tool. Know cache-aside, write-through, TTL, and the three cache pitfalls.
- **CDNs** serve static content from edge nodes close to users.
- **Read replicas** scale reads; **sharding** scales writes and storage.
- **Message queues** (Kafka) decouple producers from consumers and absorb traffic spikes.
- **Microservices** enable independent deployment and scaling, at the cost of distributed systems complexity.
- **API gateways** centralise auth, rate limiting, and routing.
- **Circuit breakers** prevent cascading failures when downstream services degrade.
- **Service discovery** (Consul, etcd, Kubernetes DNS) lets services find each other dynamically.

8.17 Database Indexing: B-Trees and LSM Trees

B-Tree Index

The workhorse of traditional relational databases (PostgreSQL, MySQL, Oracle).
A **B-tree** is a self-balancing n-ary search tree where each node holds n-1 keys and n child pointers. All leaf nodes are at the same depth. A node fits in one disk page (typically 4–16 KB); n can be several hundred.
Properties:

- Read: $O(\log N)$ disk reads, where N = number of records. Typically 3–4 levels for billions of rows.
- Write: $O(\log N)$ disk reads + 1 write (in-place update). Good for random writes.
- Range queries: excellent — leaf nodes are linked, walk the linked list.
- The tree root and upper levels are almost always in the OS page cache (hot pages), so most reads require only 1–2 actual disk reads.

LSM Tree (Log-Structured Merge-Tree)

Used by: LevelDB, RocksDB, Cassandra, HBase, ScyllaDB. Optimised for **high write throughput** by making writes sequential.
Write path:

1. Write to **WAL** (Write-Ahead Log): sequential disk write for durability.
2. Write to **memtable**: an in-memory sorted structure (skiplist or red-black tree).
3. When memtable reaches size limit, flush to disk as an **SSTable** (Sorted String Table): immutable, sorted, compressed file.

Compaction: background process merges SSTables (merge sort), removing deleted entries and old versions, producing fewer, larger SSTables. Keeps read amplification bounded.
Read path: Check memtable, then SSTables from newest to oldest. A Bloom filter per SSTable avoids reading SSTables that don't contain the key.

B-Tree vs. LSM Tree

Property	B-Tree	LSM Tree
Write amplification	Low (1 write per update)	High (compaction rewrites data many times)
Read amplification	Low (logarithmic)	Higher (check multiple SSTables)
Space amplification	Low	Moderate (stale data until compaction)
Write throughput	Moderate	Very high (sequential writes)
Random writes	Disk-seek limited	Very fast (goes to memtable)
Range reads	Excellent	Good (sorted SSTables)

8.18 Write-Ahead Logging (WAL)

WAL

Before modifying any data page, write a description of the change to a **sequential log file** on disk (the WAL). The log entry must be durable (fsynced) before the original data page is modified.
Why? Disk writes are not atomic. A power failure mid-write can leave a data page in a corrupt, partially-written state. The WAL allows recovery:

- On crash: re-read the WAL from the last checkpoint. **Redo** committed transactions (re-apply their WAL entries). **Undo** uncommitted transactions (reverse their partial changes).
- The data pages on disk may be stale — that's fine, the WAL contains the truth.

WAL writes are sequential (fast on HDD, also fast on SSD). Data page writes can be batched and written asynchronously.

WAL and Group Commit

The expensive part of WAL is `fsync()` (flush to durable storage). **Group commit**: instead of fsyncing after every transaction, batch multiple transactions' WAL records and fsync once. All transactions in the batch are durable after one fsync. Dramatically reduces the fsync rate.
PostgreSQL `commit_delay` and MySQL's `sync_binlog` control this trade-off.

8.19 MVCC — Multi-Version Concurrency Control

MVCC

Traditional locking: readers block writers, writers block readers. This kills concurrency.
MVCC maintains **multiple versions** of each row. Writers create new versions without overwriting old ones. Readers see the version that was current at the start of their transaction — no blocking.
How it works (PostgreSQL):

- Every row has **xmin** (transaction ID that created it) and **xmax** (transaction ID that deleted/updated it, or 0).
- A transaction sees rows where **xmin** was committed before the transaction started and **xmax** is either 0 or committed *after* the transaction started.
- Update = insert new version (with current txn's xmin) + set old version's xmax.
- Old versions accumulate — **VACUUM** cleans them up asynchronously.

Used by: PostgreSQL, MySQL InnoDB, Oracle, CockroachDB, SQL Server.

MVCC Isolation Level Example

```
-- Two concurrent transactions
-- T1 starts at time 100 and reads
-- T2 starts at time 101 and writes

-- T2 (time 101): UPDATE accounts SET balance = 500 WHERE id = 1;
-- Creates new row version (xmin=101) and marks old version (xmax=101)

-- T1 (time 100): SELECT balance FROM accounts WHERE id = 1;
-- Sees the old version (xmin < 100, xmax=101 > 100) -- reads old balance
-- T1 is completely unblocked by T2's write -- Repeatable Read isolation
```

8.20 Connection Pooling

Why Connection Pooling?

Establishing a database connection is expensive (∼10–100 ms): TCP handshake, TLS negotiation, authentication, session setup. An application handling 10,000 RPS cannot create a new connection per request.
A **connection pool** maintains a set of pre-established, reusable connections. When a request needs a DB connection: check out one from the pool (fast, ∼1 µs). When done: return it to the pool (not close it).

Pool Sizing

A common mistake: making the pool too large. Each DB connection consumes: ∼5–10 MB on the server side, plus a backend process (PostgreSQL forks a process per connection). 1,000 connections = 5–10 GB RAM on the DB server.
The Hikari CP author's rule of thumb:

$$\text{pool size} = (N_{\text{cores}} \times 2) + N_{\text{effective\_spindle\_disks}}$$

For a 4-core DB server with SSDs: pool size ≈ 9. Sounds tiny, but DB servers are I/O bound, not CPU bound; more connections cause context-switch overhead.
Use **PgBouncer** (connection pooler for PostgreSQL) to multiplex thousands of application connections onto a small DB pool.

8.21 Backpressure

Backpressure

When a producer generates work faster than a consumer can process it, the queue grows without bound — eventually causing OOM or cascading failures. The correct response is **backpressure**: signal to the producer to slow down.

Mechanisms:

- **Blocking queue**: producer's `put()` blocks when the queue is full. Simple and effective. Works in-process.
- **TCP flow control**: the receiver's window size communicates buffer capacity back to the sender. The kernel handles this automatically.
- **HTTP/2 flow control**: per-stream and per-connection flow control windows.
- **Reactive Streams / Akka Streams**: explicit demand signalling. The consumer requests n items; the producer sends at most n.
- **Rate limiting at the source**: the producer checks the queue depth and throttles before it fills.

Lack of Backpressure = Cascading Failure

Without backpressure:

1. Client sends 100K RPS; server handles 50K RPS.
2. Queue fills up. Memory exhausted. OOM killer fires.
3. Server restarts (during which 0 RPS served).
4. Clients retry immediately — 200K RPS burst.
5. Cycle repeats — **retry storm**.

With backpressure: server returns HTTP 429 when overloaded. Clients back off. System degrades gracefully instead of crashing.

8.22 Retry Strategies: Exponential Backoff and Jitter

Exponential Backoff

On failure, don't retry immediately. Wait $\min(\text{cap}, \text{base} \times 2^n)$ before the n-th retry:

```python
import time, random

def retry_with_backoff(fn, max_retries=5, base=0.5, cap=30.0):
    for attempt in range(max_retries):
        try:
            return fn()
        except Exception:
            if attempt == max_retries - 1: raise
            delay = min(cap, base * (2 ** attempt))
            # Add full jitter to prevent thundering herd
            delay = random.uniform(0, delay)
            time.sleep(delay)
```

Jitter: Preventing the Thundering Herd

Without jitter: if 10,000 clients all fail at the same time (e.g., server restarts), they all back off for exactly 2 seconds and retry simultaneously — causing another overload spike. **Full jitter** (`random.uniform(0, delay)`) spreads retries over time, smoothing the load.

AWS's blog on backoff and jitter recommends **decorrelated jitter**:

$$\text{sleep} = \min(\text{cap}, \text{random}(\text{base}, \text{prev\_sleep} \times 3))$$

which produces better-distributed retry intervals than simple exponential + full jitter.

8.23 Observability: Metrics, Logs, and Traces

The Three Pillars of Observability

- **Metrics**: numerical measurements over time (RPS, p99 latency, error rate, CPU usage, GC pause time). Low storage overhead; ideal for alerting and dashboards. Tools: Prometheus + Grafana, Datadog, CloudWatch.
- **Logs**: timestamped, unstructured or structured records of discrete events. High storage overhead; ideal for debugging specific errors. Tools: ELK Stack (Elasticsearch + Logstash + Kibana), Loki, Splunk.
- **Distributed Traces**: end-to-end recording of a request as it flows through multiple services. Each service contributes **spans** (start time, duration, metadata) linked by a **trace ID**. Tools: Jaeger, Zipkin, AWS X-Ray, Honeycomb.

OpenTelemetry

OpenTelemetry (OTel) is the emerging industry standard for instrumentation: a single SDK that emits metrics, logs, and traces in a vendor-neutral format.

```python
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider

tracer = trace.get_tracer("my-service")

def process_order(order_id: str):
    with tracer.start_as_current_span("process_order") as span:
        span.set_attribute("order.id", order_id)
        # Span automatically includes: start time, end time, parent trace ID
        result = fetch_inventory()          # auto-propagates trace context
        charge_payment()                    # each inner call creates a child span
        span.set_attribute("order.total", result.total)
```

8.24 SLIs, SLOs, and SLAs

SLI, SLO, SLA

- **SLI** (Service Level Indicator): a quantitative measure of service behaviour. Examples: request success rate, p99 latency, availability (uptime fraction).
- **SLO** (Service Level Objective): a target value for an SLI. Example: "99.9% of requests complete in <200 ms over a rolling 30-day window." SLOs are internal targets.
- **SLA** (Service Level Agreement): a contractual commitment between provider and customer, with penalties for violation. SLAs are weaker than SLOs (you'd be penalised if you miss the SLA; the SLO gives buffer before that happens).

Error Budgets

SRE concept (Google's Site Reliability Engineering): if your SLO is 99.9% availability, you have $0.1\% \times 30$ days ≈ 43 minutes of downtime budget per month.
If you've spent your error budget, reliability work takes priority over new features. If you have budget remaining, you can take more risks. This creates a data-driven conversation between developers (want to ship fast) and SREs (want reliability).

8.25 Blue-Green and Canary Deployments

Blue-Green Deployment

Maintain two identical production environments: **Blue** (currently live) and **Green** (idle). Deploy the new version to Green. Test thoroughly. Switch the load balancer to point to Green in one atomic operation.
Rollback: switch load balancer back to Blue — instant. No re-deployment needed.
Limitation: requires 2x infrastructure. Database migrations must be backward-compatible (both Blue and Green point to the same database).

Canary Deployment

Gradually roll out a new version to a **small percentage** of traffic (e.g., 1%), monitor metrics (error rate, latency, business metrics), then gradually increase (5% -> 25% -> 50% -> 100%) if all looks good.
Advantages: real production traffic exposes real bugs. Blast radius limited to the canary percentage. Rollback: shift 0% traffic to the canary.
Feature flags complement canary deployments: code ships to all servers but is only enabled for specific users (by user ID, location, account tier). Enables instant rollback without redeployment and A/B testing.

Chapter 8 Additional — Key Takeaways

- **B-trees**: $O(\log N)$ reads and random writes, great for OLTP. **LSM trees**: high write throughput via sequential I/O, read-amplification managed by Bloom filters and compaction.
- **WAL** + group commit = durability with high write throughput. Sequential log writes are fast; data pages written lazily.
- **MVCC**: readers never block writers. Multiple row versions enable snapshot isolation. VACUUM cleans old versions.
- **Connection pool sizing**: smaller than you think. 2×cores + disks. Use PgBouncer to multiplex app connections onto a small DB pool.
- **Backpressure** prevents retry storms and cascading failure. Implement at every I/O boundary.
- **Exponential backoff + full jitter** prevents thundering herds on recovery.
- **Observability** = metrics (Prometheus) + logs (Loki/ELK) + traces (Jaeger). Use OpenTelemetry for vendor-neutral instrumentation.
- **SLO + error budget**: data-driven reliability decision-making.
- **Canary deployments** + feature flags: ship safely, rollback instantly.